

Model-Based Systems Engineering

Methods, Languages, Tools, and Integration Patterns

A Comprehensive Technical Reference

Covering Pareto Optimality, Response Surface Methods, MBSE Simulation, SysML v1/v2, Modelica, AADL, FMI Co-Simulation, Thermal Analysis, Controls Integration, and Hardware-in-the-Loop Verification

Executive Summary.....	4
1. Comparative Overview: Three Foundational Methods.....	5
1.1 What Each Method Answers.....	5
1.2 How They Interact.....	5
2. Response Surface Methods (RSM).....	7
2.1 The Mathematical Foundation.....	7
2.2 Design of Experiments Strategies.....	7
2.3 When RSM Is the Right Tool.....	7
2.4 Industrial Applications.....	8
2.5 Limitations.....	8
3. Pareto Optimality Analysis.....	9
3.1 The Dominance Concept.....	9
3.2 Constructing the Frontier.....	9
3.3 The Shape of the Frontier and Its Meaning.....	9
3.4 Selecting from the Frontier.....	10
3.5 Multi-Criteria Decision Analysis for High-Dimensional Problems.....	10
4. MBSE Simulations: Architecture and Scope.....	11
4.1 The MBSE Stack.....	11
4.2 The V-Model and Simulation's Role.....	11
5. SysML: From v1 to v2.....	13
5.1 SysML v1: Strengths and Limitations.....	13
5.2 SysML v2: Architectural Rethink.....	13
5.3 v1 vs. v2: Feature Comparison.....	14
5.4 Key New Features in SysML v2.....	15
6. Modelica: Equation-Based Multi-Domain Simulation.....	16
6.1 The Acausal Architecture.....	16
6.2 Domain Coverage.....	16
6.3 Where Modelica Excels.....	16
6.4 Tooling.....	17
6.5 Limitations.....	17
7. AADL: Architecture Analysis for Safety-Critical Systems.....	19
7.1 The Fundamental Capability: Schedulability Analysis.....	19
7.2 Component Taxonomy.....	19
7.3 Analysis Capabilities.....	20
7.4 When to Use AADL Instead of SysML.....	20
7.5 Combining AADL and SysML in Mature Programs.....	20
8. The Functional Mockup Interface (FMI): Simulation Interoperability.....	21
8.1 The Functional Mockup Unit (FMU).....	21
8.2 Two Interface Modes.....	21
8.3 The Master Algorithm.....	21
8.4 What Gets Wrapped as FMUs.....	22

8.5 The Communication Step Size Trade-Off.....	22
9. Multi-Domain Co-Simulation.....	23
9.1 Why Single-Domain Simulation Is Insufficient.....	23
9.2 The Multi-Domain Aircraft ECS Example.....	23
9.3 Stability and Accuracy.....	23
10. Thermal Simulation Integration.....	25
10.1 Tool Landscape.....	25
10.2 The Four MBSE Integration Points.....	25
10.3 Thermo-Mechanical Coupling.....	25
10.4 Reduced-Order Models for System Integration.....	26
11. Controls Simulation and Architecture Integration.....	27
11.1 The Abstraction Mismatch.....	27
11.2 The Plant Model Evolution.....	27
11.3 Integration Mechanisms.....	27
11.4 Control Architecture Patterns.....	28
11.5 The Verification Progression: MiL to HiL.....	28
12. Hardware-in-the-Loop Verification.....	29
12.1 Physical Architecture of a HiL Rig.....	29
12.2 What HiL Uniquely Reveals.....	29
12.3 HiL in the Certification Evidence Chain.....	30
12.4 HiL in the MBSE Knowledge Graph.....	30
13. The Integrated MBSE Architecture.....	31
13.1 The Complete Integration Pipeline.....	31
13.2 The Knowledge Graph as Traceability Backbone.....	31
13.3 The Circuit Breaker Pattern for AI Grounding.....	32
14. Conclusions and Practitioner Guidance.....	33
14.1 Method Selection Guidance.....	33
14.2 Common Integration Failure Modes.....	33
14.3 The Path Forward.....	34
Appendix A: Quick Reference — Key Acronyms and Terms.....	35

Executive Summary

This document provides a comprehensive technical reference for the core methods, modeling languages, simulation tools, and integration patterns that constitute modern Model-Based Systems Engineering (MBSE) practice. It is organized as a coherent progression from foundational decision-support methods through architectural modeling languages, physics simulation, cross-domain integration, and verification.

Three foundational analytical methods are examined in depth: Pareto Optimality Analysis, Response Surface Methods (RSM), and MBSE Simulations. These methods are not independent alternatives — they form a complementary triad in which RSM generates efficient surrogate models from expensive simulations, MBSE provides the structural architectural grounding, and Pareto analysis characterizes the multi-objective trade-off space for stakeholder decision-making.

The MBSE simulation ecosystem is explored through its constituent layers: SysML v1 and v2 as the architectural modeling languages, Modelica as the equation-based multi-domain physics simulation language, AADL as the safety-critical embedded systems architecture language, and the Functional Mockup Interface (FMI) standard as the interoperability fabric connecting them. Domain-specific integration challenges are then examined for thermal simulation, controls simulation, and hardware-in-the-loop verification, closing with a discussion of how knowledge graph architectures can ground AI assistance against authoritative engineering models.

Central thesis: The most significant frontier in engineering simulation is not fidelity within any single domain, but semantic integration across domains — making the connections between architectural intent, physics simulation, verification evidence, and stakeholder requirements first-class, queryable artifacts rather than documents maintained by hand.

PART I

Foundational Decision-Support Methods

1. Comparative Overview: Three Foundational Methods

Before examining each method in depth, it is essential to understand how they relate to one another architecturally. Pareto Optimality Analysis, Response Surface Methods, and MBSE Simulations are not competing approaches to the same problem. They address different questions, operate at different levels of abstraction, and are most powerful when used in concert.

1.1 What Each Method Answers

Response Surface Methods (RSM) answer: "How does system behavior vary across my design parameter space, and where is the optimum?" RSM compresses expensive experimental or simulation effort into a polynomial surrogate model that can be swept rapidly to find performance peaks.

MBSE Simulations answer: "What will this system actually do, as a complete integrated entity, when all physics and behaviors interact?" MBSE simulation connects structural architecture to executable physics models, enabling verification that no single-domain tool can provide.

Pareto Optimality Analysis answer: "Given multiple conflicting objectives, what is the set of solutions from which no stakeholder decision can be deferred to mathematics?" Pareto analysis does not find the optimal solution — it characterizes the trade-off space from which humans must choose.

1.2 How They Interact

The three methods form a natural pipeline in mature systems engineering programs. MBSE simulations provide the high-fidelity physics ground truth. RSM samples that simulation at strategically chosen operating points using Design of Experiments (DoE) to build a computationally cheap surrogate. The surrogate is then swept by a multi-objective optimization algorithm to generate the Pareto frontier. Stakeholders use the frontier to make explicit trade-off decisions informed by the full fidelity of the underlying simulation.

Dimension	RSM	Pareto Optimality	MBSE Simulation
Primary question	Where is the optimum in parameter space?	What trade-offs are unavoidable?	What does the system actually do?
Abstraction	Polynomial surface approximation	Solution space geometry	Full system model with physics
Primary output	Surrogate function $\hat{y} = f(x)$	Pareto frontier set	Time-history simulation artifacts

Compute cost	Low (after fitting)	Moderate to high	High
Key limitation	Valid only within design space	Requires well-defined objectives	Model fidelity vs. calibration gap
When it fails	Highly nonlinear / discontinuous systems	Poorly defined or too many objectives	Model drift from physical reality
Feeds into	Pareto optimization (surrogate sweep)	Stakeholder decision process	RSM sampling, Pareto objectives

2. Response Surface Methods (RSM)

RSM is fundamentally a strategy for making expensive systems cheap to explore. The core insight is simple: if running one simulation or experiment costs significant time or money, exhaustive sweeping of the full design space is impractical. RSM runs a small, strategically chosen set of experiments, fits a smooth mathematical surface to the results, and then optimizes the surface instead of the real system.

2.1 The Mathematical Foundation

The surface fitted by RSM is a second-order polynomial. For two factors x_1 and x_2 , the model is:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_{12} x_1 x_2 + \beta_{11} x_1^2 + \beta_{22} x_2^2$$

The six terms capture: a constant intercept, two linear main effects, one two-way interaction, and two pure quadratic curvature terms. With three factors the model has 10 terms; with four factors, 15. The β coefficients are estimated by ordinary least squares once experimental data is collected. The DoE design determines the matrix of factor settings, and a well-conditioned design matrix ensures small estimation errors and good predictive power.

The curse of dimensionality limits RSM applicability. The number of model terms grows as $(k+1)(k+2)/2$ where k is the number of factors. Beyond 6–7 factors, run counts become prohibitive and practitioners typically employ machine-learning surrogates such as Gaussian Process regression (Kriging) instead.

2.2 Design of Experiments Strategies

2.2.1 Central Composite Design (CCD)

CCD is the workhorse RSM design. It combines three types of points: factorial core points at the corners of the design space, center points (multiple runs at the origin providing pure error and curvature estimates), and axial or "star" points extending beyond the factorial region along each factor axis. The star points provide the information needed to estimate quadratic terms efficiently.

For k factors, a CCD requires $2k$ (factorial) + $2k$ (axial) + n_0 (center) runs. For three factors: $8 + 6 + 6 = 20$ runs, compared to 125 for a full 5-level factorial.

2.2.2 Box-Behnken Design

Box-Behnken designs avoid extreme corner and axial points, placing all runs at midpoints of the design cube's edges plus center points. This is attractive when extreme factor combinations are physically dangerous or practically impossible — such as maximum temperature combined with maximum pressure combined with maximum catalyst loading. For three factors, Box-Behnken requires only 13 runs versus CCD's 20, at the cost of slightly lower predictive efficiency near the design space boundaries.

2.3 When RSM Is the Right Tool

RSM earns its place when three conditions align simultaneously:

- **Smooth, continuous response:** The system behavior can be approximated by a low-order polynomial. Systems with discontinuities, phase transitions, or highly multimodal responses violate this assumption.
- **Pre-screened, bounded factors:** The modeler knows which variables matter (screening is complete) and has a defined region of interest. RSM is not a screening tool.
- **Expensive evaluations:** Each run costs enough that a designed experiment's efficiency gain is meaningful — CFD runs, physical experiments, high-fidelity FEA analyses, pharmaceutical manufacturing trials.

2.4 Industrial Applications

Pharmaceutical process development is a canonical RSM domain. ICH Q8/Q9/Q10 Quality by Design guidance explicitly endorses RSM for establishing the design space — the region within which process parameters can move without negatively affecting product quality attributes. The Pareto frontier enters when multiple quality attributes are in tension: high dissolution rate versus low friability versus acceptable blend uniformity.

Aerospace and defense programs use RSM for aerodynamic shape optimization, propulsion parameter studies, and structural weight-versus-strength trade-offs. When a CFD analysis run costs 8 hours on a 32-core cluster, DoE sampling of the design space followed by surrogate optimization reduces thousands of potential design evaluations to dozens of actual runs.

2.5 Limitations

- **Validity boundary:** The surrogate is only reliable within the design space where experiments were run. Extrapolation produces unreliable predictions.
- **Stationarity assumption:** RSM assumes the input-output relationship does not change over time or depend on factors not included in the model.
- **Dimensionality ceiling:** Practical limit of approximately 6-7 factors before run counts or the number of model terms become prohibitive.
- **Silent failure:** A polynomial surface fitted to insufficient data may appear statistically valid while predicting the optimum in entirely the wrong location. Lack-of-fit testing and confirmation runs are essential validation steps.

3. Pareto Optimality Analysis

Pareto optimality is the mathematical formalization of an uncomfortable truth: in any multi-objective problem, there is no single "best" solution. Instead there is a set of solutions that are each unbeatable in their own way — you cannot improve any one objective without degrading at least one other. That set is the Pareto frontier, and it is the correct answer to a multi-objective optimization problem. Everything beyond characterizing the frontier requires human judgment about values, priorities, and acceptable trade-offs.

3.1 The Dominance Concept

The operational definition of Pareto optimality rests on the concept of dominance. Solution A dominates solution B if A is at least as good as B on every objective and strictly better on at least one. A solution is Pareto-optimal if and only if nothing dominates it. The Pareto frontier is the complete set of non-dominated solutions.

This definition has an important implication: the frontier is completely determined by the objective functions and the candidate solution set. It does not depend on weights, preferences, or stakeholder values. Those inputs are required only to select a specific solution from the frontier — not to compute the frontier itself. This separation of computation from preference is what makes Pareto analysis a principled approach to multi-stakeholder decision-making.

3.2 Constructing the Frontier

3.2.1 For Discrete Solution Sets

For a finite set of candidate solutions, frontier construction is conceptually straightforward: test every pair for dominance and retain only the non-dominated solutions. The naive algorithm is $O(n^2 \cdot k)$ where n is the number of solutions and k is the number of objectives. More efficient algorithms (such as the Fast Non-dominated Sort used in NSGA-II) achieve $O(n \log n)$ for two objectives and $O(n \cdot \log^{k-2}(k-2) \cdot n)$ for higher dimensions.

3.2.2 For Continuous Design Spaces

When searching an infinite design space, constructing the frontier requires multi-objective evolutionary optimization algorithms. The dominant family is EMO (Evolutionary Multi-objective Optimization). NSGA-II (Non-dominated Sorting Genetic Algorithm II) is the most widely used algorithm for two- and three-objective problems. It maintains a population of candidate solutions, selects parents based on Pareto rank and crowding distance (a spacing metric that preserves diversity along the frontier), and generates offspring through crossover and mutation.

NSGA-III extends the approach to four or more objectives using reference point-based selection instead of crowding distance, which scales better to high-dimensional objective spaces.

When each objective evaluation requires an expensive simulation, RSM surrogates replace the direct simulation calls. The optimizer sweeps the cheap polynomial surface instead of the high-fidelity model, and the resulting frontier candidates are then verified against the full simulation.

3.3 The Shape of the Frontier and Its Meaning

The frontier's geometry carries engineering information that practitioners should read actively.

- **Convex frontier:** Trade-offs are smooth and gradual. Moving along the frontier, you lose one objective and gain another at a roughly constant rate. Weighted-sum optimization reliably finds the correct solution.
- **Non-convex frontier:** Trade-offs are asymmetric or discontinuous. Certain regions are unreachable by weighted-sum methods. Reference point methods or epsilon-constraint approaches are required.
- **Knee point:** The point of maximum curvature — where giving up a small amount on one objective yields the largest gain on the other. This is often the engineering default when stakeholder preferences cannot be articulated a priori.
- **Discontinuous frontier:** Gaps signal distinct design paradigms that do not blend smoothly — for example, composite versus metallic structural concepts where hybrid designs are inferior to either pure approach.

3.4 Selecting from the Frontier

The frontier does not make decisions — it structures the decision conversation. Standard selection approaches include:

Weighted sum scalarization: Assign weights reflecting stakeholder priorities, compute a weighted score for each frontier point, select the maximum. Simple but requires weights to be specified before seeing the frontier, and cannot reach non-convex regions.

Reference point methods: The decision-maker specifies aspiration levels (I need at least 85% reliability and at most \$2M cost), and the algorithm finds frontier points closest to that reference. More natural for engineers who think in requirements rather than abstract weights.

Knee point selection: Algorithmic selection of the frontier point with maximum curvature. The engineering default when stakeholder preferences cannot be articulated — it represents maximum simultaneous value across all objectives.

3.5 Multi-Criteria Decision Analysis for High-Dimensional Problems

With two objectives the frontier is a curve; with three, a surface. Beyond three objectives — weight, cost, reliability, performance, manufacturability, maintainability — the frontier becomes a high-dimensional manifold that requires specialized Multi-Criteria Decision Analysis (MCDA) methods. TOPSIS, VIKOR, ELECTRE, and PROMETHEE impose structure on the decision space through pairwise comparisons and preference modeling.

The practical implication: Pareto analysis is most powerful when upstream work has identified which 2-3 objectives are genuinely in tension. This requires stakeholder interviews, functional decomposition, and the kind of semantic modeling that clarifies which performance attributes are truly independent versus correlated — precisely the work that a well-constructed MBSE model facilitates.

PART II

The MBSE Simulation Ecosystem

4. MBSE Simulations: Architecture and Scope

Model-Based Systems Engineering simulation rests on a core idea: replace documents as the primary system artifact with a computable, semantically rich model that can be queried, verified, simulated, and evolved. The model is the truth — not the Word document that describes it.

MBSE simulation is not a single tool but an ecosystem organized in layers: architectural modeling languages that capture system structure and behavior, physics simulation languages that capture multi-domain dynamics, integration standards that enable co-simulation across tools, and verification environments that close the loop between simulation results and architectural requirements.

4.1 The MBSE Stack

Layer	Purpose	Key Technologies
Architectural modeling	Structure, behavior, requirements, allocations	SysML v1/v2, AADL
Physics simulation	Multi-domain continuous-time dynamics	Modelica, Simulink, ANSYS
Integration standard	Cross-tool co-simulation interface	FMI/FMU, SysML v2 API
Verification	Requirements closure, test execution	MiL, SiL, PiL, HiL testing
Knowledge management	Traceability, provenance, AI grounding	Knowledge graphs, REST APIs

4.2 The V-Model and Simulation's Role

The V-model organizes MBSE simulation activities across the system development lifecycle. The left side of the V descends from system requirements through subsystem design to detailed implementation. The right side ascends through corresponding verification and validation activities. Simulation methods map to specific points on this V:

- **System architecture (top left):** SysML v2 captures requirements, functional allocation, and interface definitions. Pareto analysis informs architecture selection.
- **Subsystem design (middle left):** Modelica and Simulink models instantiate subsystem physics. RSM builds surrogates for expensive subsystems.

- **Implementation (bottom):** Code generation from architectural models (AADL/TASTE) or control law implementation in Simulink.
- **Software verification (middle right):** MiL and SiL testing verify control algorithm behavior in simulation.
- **Subsystem verification (upper right):** HiL testing verifies the real controller in a simulated plant environment.
- **System validation (top right):** Full vehicle or system integration testing closes requirements.

5. SysML: From v1 to v2

SysML (Systems Modeling Language) is the dominant architectural modeling language for MBSE. The transition from v1 to v2, completed with the OMG standard finalization in 2024, represents not an incremental update but a fundamental architectural rethink driven by a decade of hard-won lessons about what broke in practice.

5.1 SysML v1: Strengths and Limitations

SysML v1 was designed in 2006 as a profile on top of UML 2 — a pragmatic choice that leveraged UML's existing tooling ecosystem. It introduced nine diagram types addressing requirements, structure, and behavior, and established systems modeling as a distinct discipline with formal tool support.

Mature ecosystem: Cameo Systems Modeler, IBM Rhapsody, Enterprise Architect, and Capella all provide robust SysML v1 support with large installed bases and trained practitioner communities.

The UML inheritance problem: UML was designed for software systems. Its metamodel has concepts — classes, interfaces, components — that map awkwardly onto physical systems with multiple physical domains, dimensional quantities, and mathematical behavior. Every systems modeler who has tried to represent a thermal-hydraulic system in SysML v1 hits the same wall: the language does not natively understand units, quantities, or physics.

Graphical-first architecture: When diagrams are the primary artifact, the model is the picture. Pictures do not compose, version-control cleanly, transform automatically, or feed into downstream analysis tools without manual intervention. Every diagram a systems engineer drew in Cameo was a hand-crafted visualization of a model living in a proprietary XML file that only Cameo could faithfully read.

5.2 SysML v2: Architectural Rethink

SysML v2 addresses v1's foundational limitations through three architectural commitments: a new semantic foundation (KerML), a normative textual syntax, and a standardized REST API for model repositories.

5.2.1 KerML: The Semantic Foundation

KerML (Kernel Modeling Language) is the semantic core of SysML v2, replacing UML as the foundation. Its key architectural move unifies classifier, feature, and relationship into a single coherent metamodel without UML's baggage. Everything is either a type (something that classifies instances) or a feature (something that characterizes instances of a type).

KerML introduces occurrence semantics: every element exists over a span of time and has a life history. A PartUsage is not just a structural placeholder — it is an entity that starts, persists, changes state, and ends. This temporal semantics is the bridge to simulation: connecting the structural model to a simulation engine becomes a mapping rather than a translation.

5.2.2 Textual Syntax: Models as Text

SysML v2 ships a normative textual notation. A part definition looks like:

```
part def Vehicle {  
    part engine : Engine;
```

```
part transmission : Transmission;
attribute mass : MassValue;
attribute topSpeed : SpeedValue;
}
```

This enables models to live in Git repositories, be diffed line-by-line, be generated by scripts, be validated by CI pipelines, and be written in any text editor. The VSCode SysIDE extension provides syntax highlighting and validation. This is the infrastructure-as-code moment for systems engineering — the same philosophical shift that moved cloud infrastructure from GUI-managed to Terraform-managed, version-controlled state.

5.2.3 Strong Typing with Physical Units

SysML v2 ships standard libraries conformant to ISO 80000 defining quantity kinds and value types. Dimensional analysis becomes model-level:

```
attribute mass : MassValue = 1500 [kg];
attribute velocity : SpeedValue = 27.8 [m/s];
attribute kineticEnergy : EnergyValue = 0.5 * mass * velocity**2;
```

A conformant tool can now catch a modeler who adds a mass to a force, or assigns a temperature where a pressure is expected. For complex multi-domain systems, this prevents a whole class of interface errors that currently propagate silently through v1 models into simulation setups.

5.2.4 The REST API: Federated Model Repository

SysML v2 specifies not just the language but the infrastructure for storing and exchanging models. The API defines standard endpoints for project management, element CRUD operations, relationship traversal, and query execution, organized around a Git-like commit model.

```
GET /projects/{projectId}/commits/{commitId}/elements
POST /projects/{projectId}/commits
GET /projects/{projectId}/commits/{commitId}/elements/{elementId}/relationships
```

Every model element is a JSON object with a stable UUID, a type referencing the KerML metamodel, and a set of typed properties. The commit-based version model mirrors Git semantics — every change creates an immutable commit preserving full history, branches are first-class concepts, and multi-vendor tool interoperability becomes architecturally achievable rather than aspirational.

5.3 v1 vs. v2: Feature Comparison

Dimension	SysML v1	SysML v2
Foundation	UML 2 profile	KerML — purpose-built semantic core
Syntax	Graphical only; no normative text	Normative textual notation + graphical

Tool interoperability	XMI file exchange — brittle	REST API — federated, tool-neutral
Diagram types	9 fixed diagram types	Views and viewpoints — stakeholder-tailored
Requirements	Text in a box; documentary only	Formal constraints; machine-verifiable
Units / typing	Weak typing inherited from UML	ISO 80000 units; dimensional analysis
Simulation bridge	Parametric diagram — limited	Occurrence + action semantics — native
Maturity	Mature; large installed base	Standard 2024; tooling emerging

5.4 Key New Features in SysML v2

Beyond the architectural changes, several specific features of SysML v2 deserve attention:

Formal requirements with satisfaction conditions: RequirementUsage elements can carry formal constraint expressions evaluable by analysis tools, enabling model-based verification evidence rather than document-based evidence.

Action semantics: Executable behavioral semantics built into the specification — control flow, data flow, conditional branching, and iteration all have precise computational meanings, enabling direct simulation without proprietary plugins.

Views and viewpoints: Replacing fixed diagram types with a stakeholder-driven projection mechanism. A viewpoint specifies what a particular stakeholder role needs to see; a view renders the model filtered through that viewpoint.

Standard libraries: SI Units, Geometry, Sequence Functions, and domain libraries ship as part of the specification — not tool add-ons. Two organizations using different conformant tools share the same definitions of MassValue and ForceValue.

6. Modelica: Equation-Based Multi-Domain Simulation

Modelica occupies a distinctive niche in the simulation landscape: it is the language of choice when a system spans multiple physical domains simultaneously and you care about the continuous-time dynamics of the whole. Its central architectural innovation — acausal, equation-based modeling — distinguishes it from every other mainstream simulation environment.

6.1 The Acausal Architecture

Every mainstream simulation environment except Modelica is causal: blocks are wired together in signal flow direction, with explicit input-output assignment. Modelica is acausal: equations describe physical relationships, and the compiler determines causality automatically through symbolic manipulation.

Consider a resistor. A causal tool requires the modeler to choose: "Is voltage the input and current the output, or vice versa?" This decision must be made before knowing how the resistor will be used in a circuit. In Modelica:

```
model Resistor
  parameter Real R = 100 "Resistance [Ohm]";
  Modelica.Electrical.Analog.Interfaces.Pin p, n;
equation
  p.v - n.v = p.i * R;
end Resistor;
```

The compiler analyzes the full system of equations, determines which variables are unknown at each point, and selects the computational sequence automatically. The same resistor model works whether it is the driven element or the driving element — causality is determined by context, not by the component definition.

6.2 Domain Coverage

The Modelica Standard Library (MSL) ships with every Modelica tool and provides approximately 1,400 model components across seven primary domains:

- **Mechanics:** Translational and rotational mechanical systems, multi-body dynamics, gear trains, spring-damper systems
- **Electrical:** Analog and digital circuits, machines, power electronics (behavioral models)
- **Thermal:** Heat transfer by conduction, convection, and radiation; thermal networks
- **Fluid:** One-dimensional pipe flow, heat exchangers, pumps, valves, open and closed circuits
- **Magnetic:** Electromagnetic devices, transformers, reluctance models
- **Controls:** PID controllers, transfer functions, state-space systems, logical blocks
- **Math:** Vector and matrix operations, nonlinear functions, interpolation tables

6.3 Where Modelica Excels

Modelica is the tool of choice for systems where domain interactions are the engineering problem, not an incidental complication. Canonical application domains include:

Automotive powertrain: BMW, Toyota, Daimler, and virtually every major OEM use Dymola or OpenModelica for vehicle system simulation. Battery thermal management, electric motor electromagnetic-mechanical coupling, gear train efficiency, and control system behavior interact in ways that only an acausal multi-domain solver handles correctly.

Building energy systems: Lawrence Berkeley National Laboratory's Buildings library is used by the US Department of Energy for HVAC optimization, district energy modeling, and building controls research. Occupancy patterns, heating loads, electrical demand, and controls interact continuously.

Process engineering: Dynamic simulation of chemical plants, nuclear thermal power systems, and oil and gas facilities where reaction kinetics, fluid thermodynamics, and heat integration must be solved simultaneously.

Aerospace ECS and hydraulics: Airbus and DLR have been major Modelica contributors. Environmental control systems, hydraulic actuation, fuel systems, and environmental control all benefit from the acausal multi-domain approach.

6.4 Tooling

Tool	Type	Primary Strength
Dymola	Commercial (Dassault)	Most robust compiler; aerospace/automotive standard
OpenModelica	Open source	Free; growing capabilities; research use
Modelon Impact	Commercial (Modelon)	Cloud-native; FMI export focus; library marketplace
Optimica Compiler Toolkit	Commercial	Optimization integration; MPC design
MATLAB/MapleSim	Commercial	Modelica support within established ecosystems

6.5 Limitations

- **High-frequency switching:** Power electronics switching at 100kHz+ creates discontinuities that degrade Modelica solver performance. SPICE-based tools are superior for detailed switching circuit design.
- **Spatially distributed phenomena:** Modelica is fundamentally lumped-parameter or 1D distributed. Full 3D thermal or structural analysis requires FEA tools, with reduced-order models (ROMs) exported to Modelica for system-level integration.
- **Discrete-event systems:** Manufacturing lines, logistics networks, and communications protocols are poorly suited to Modelica. AnyLogic or SimPy are better choices for discrete-event simulation.
- **Initialization complexity:** Large models with many components can fail to initialize with cryptic error messages. Determining sufficient initial conditions for a well-posed DAE requires Modelica expertise and is a genuine barrier for new users.

- **Solver expertise required:** DAE stiffness, solver selection, and tolerance configuration require understanding of numerical methods. A poorly configured solver produces wrong answers quietly — the most dangerous simulation failure mode.

7. AADL: Architecture Analysis for Safety-Critical Systems

AADL (Architecture Analysis and Design Language) is a standards-based modeling language developed by SAE International (AS5506) for a specific problem that SysML handles poorly: formally specifying and analyzing the runtime architecture of embedded, safety-critical software-hardware systems with enough precision that timing, scheduling, safety, and reliability properties can be proved rather than estimated.

The clearest way to understand AADL is through what question it answers. SysML answers "what is the system and how is it structured?" AADL answers "will this system's software execute correctly on this hardware, within these timing constraints, without violating these safety properties?"

7.1 The Fundamental Capability: Schedulability Analysis

AADL models a system as a composition of components with explicitly declared properties: execution time ranges, periods, deadlines, memory footprints, data access modes, communication protocols, and fault modes. These properties are not annotations for human readers — they are inputs to formal analysis tools.

When a thread is declared with a 20ms period, a worst-case execution time of 4ms, and a Rate Monotonic scheduler, AADL tooling computes whether the full task set is schedulable — whether all threads will meet their deadlines given the resource constraints. This is a mathematical certificate, not an estimate.

```
thread FlightControlThread
  properties
    Dispatch_Protocol    => Periodic;
    Period                => 20ms;
    Deadline              => 20ms;
    Compute_Execution_Time => 2ms .. 4ms;
    Priority               => 10;
end FlightControlThread;
```

7.2 Component Taxonomy

AADL defines three categories of components that map directly to embedded system architecture concerns:

Software components: Process (OS address space containing threads), Thread (schedulable execution unit with period and deadline), Data (shared memory with access mode declarations), and Subprogram (callable unit with stack-bound execution).

Hardware components: Processor (scheduling policy, clock rate), Bus (bandwidth, latency, protocol type), Memory (size, access time, stack and heap binding), and Device (sensors, actuators, driver interface).

Composite systems: System components that bridge software and hardware, carrying binding properties that map software components to hardware resources.

7.3 Analysis Capabilities

AADL's value proposition lives entirely in its analysis ecosystem:

- **Schedulability analysis (OSATE):** Formal proof that a task set meets all deadlines under Rate Monotonic Analysis (RMA) or Earliest Deadline First (EDF) scheduling.
- **End-to-end latency analysis:** Worst-case bounds on data flow from sensor input to actuator output, critical for avionics and automotive safety functions.
- **Safety analysis (Error Model Annex EMV2):** Fault tree generation and failure propagation analysis directly from the architectural model, producing certification-quality evidence.
- **Code generation (TASTE/Ocarina):** Deployment infrastructure in Ada or C generated from the architecture, so engineers write only application logic — the scaffolding derives from the model.

7.4 When to Use AADL Instead of SysML

Situation	Preferred Language	Reason
Need to prove hard real-time deadlines	AADL	Formal schedulability analysis with mathematical certificates
Partitioning IMA avionics architecture	AADL	SW/HW binding model matches ARINC 653 partitioning concepts
DO-178C certification evidence strategy	AADL	OSATE analysis reports accepted by FAA/EASA
Mission/system-level function allocation	SysML	Broader viewpoint coverage; multi-stakeholder requirements
Non-software system (thermal, structural)	SysML + Modelica	AADL has no multi-physics domain
Early-phase architecture exploration	SysML	Breadth and expressiveness favor early-phase work

7.5 Combining AADL and SysML in Mature Programs

The most sophisticated avionics and defense programs use both languages at different levels. SysML models the system architecture — aircraft functions, avionics requirements, allocation of functions to LRUs (Line Replaceable Units). AADL models the software architecture within each LRU — threads, processes, scheduling, and communications implementing the allocated functions. A traceability relationship connects the SysML function allocation to the AADL component that implements it, closing the verification loop.

8. The Functional Mockup Interface (FMI): Simulation Interoperability

FMI is the engineering equivalent of a USB standard — a vendor-neutral contract that lets simulation components from completely different tools plug into each other without either side needing to know anything about the other's internals. Before FMI existed, connecting a Modelica thermal model to a Simulink controls model to an ANSYS structural model required custom glue code, format translators, and fragile point-to-point integrations that broke every time a tool version changed.

The standard is maintained by the Modelica Association and has three major versions: FMI 1.0 (2010), FMI 2.0 (2014 — the widely deployed version), and FMI 3.0 (2022 — adding clocked variables, array support, and binary data).

8.1 The Functional Mockup Unit (FMU)

An FMU is a self-contained simulation component — a zip archive with a specific internal structure:

- **modelDescription.xml**: The semantic contract. Declares every variable the FMU exposes: name, data type, causality (input/output/parameter/state), variability, and physical unit. A host environment reads this XML at import time and knows exactly what the FMU expects and provides.
- **binaries/ (.dll or .so)**: The compiled simulation logic exposing a C API — standardized function calls (`fmi3SetReal`, `fmi3DoStep`, `fmi3GetReal`) that any host environment can invoke.
- **sources/**: Optional C source code enabling recompilation for different platforms.
- **resources/**: Lookup tables, maps, calibration data, and other auxiliary files required at runtime.

8.2 Two Interface Modes

Model Exchange: Exports the model's equations without a solver. The importing tool integrates those equations using its own numerical solver, enabling tighter numerical coupling and adaptive step sizing. Requires all FMUs in the configuration to share the host solver.

Co-Simulation: Packages the solver inside the FMU. Each FMU advances its own internal state from t to $t+h$ using its native solver. The master algorithm coordinates variable exchange at communication points. This preserves each tool's native solver expertise and is the mode used in the vast majority of industrial deployments.

8.3 The Master Algorithm

The master algorithm coordinates the co-simulation. At each communication point it reads outputs, distributes inputs, advances all FMUs, and collects new outputs. Three coordination schemes handle different coupling regimes:

- **Jacobi (parallel)**: All FMUs advance simultaneously using outputs from the previous communication point. Parallelizable but introduces one-step delay at every interface. Can destabilize tightly coupled systems.
- **Gauss-Seidel (sequential)**: FMUs update one at a time, passing fresh outputs immediately to downstream FMUs. Reduces delay error for asymmetrically coupled systems at the cost of sequential execution.

- **Iterative (Newton-Raphson):** Treats each communication point as a nonlinear solve, iterating until inputs and outputs converge. Recovers near-monolithic solver accuracy at the cost of running each FMU k times per step.

8.4 What Gets Wrapped as FMUs

The breadth of FMU exporters in production use is wider than commonly appreciated:

- **Modelica tools:** Dymola and OpenModelica export FMUs natively — the canonical use case.
- **Simulink:** MathWorks' FMI Kit exports Simulink models as FMUs, enabling controls models to participate in multi-domain co-simulations.
- **FEA reduced-order models:** ANSYS and Nastran can export ROMs as FMUs. A 10-million-cell structural model is compressed to a low-dimensional ROM that runs in milliseconds while retaining dominant dynamic behavior.
- **AADL/TASTE software timing models:** Software architecture timing behavior modeled in AADL can export FMUs representing computational latency and scheduling effects without requiring the actual flight software to exist.
- **Physical test rigs:** Real hardware connected through dSPACE or NI real-time systems can participate as hardware FMUs in otherwise virtual co-simulations — the foundation for hybrid HiL/SiL configurations.

8.5 The Communication Step Size Trade-Off

The communication step size h is the central numerical challenge. The governing rule of thumb: h should be at least ten times smaller than the fastest dynamic time constant at any interface. A thermal-mechanical coupling with a 1-second time constant tolerates $h = 0.1$ seconds. A fast electrical transient with a 1ms time constant requires $h = 0.1$ ms or domain decoupling.

FMI 3.0 introduced clocked variables to address sampled-data systems — digital controllers, CAN bus messages — allowing communication points to synchronize with physical clock events rather than fixed time steps.

PART III

Domain-Specific Integration Patterns

9. Multi-Domain Co-Simulation

Co-simulation is the practice of running multiple simulation models simultaneously — each solving its own physics in its own tool — while exchanging state information at regular intervals so the combined system behaves as an integrated whole. Real engineered systems do not respect domain boundaries: a jet engine is simultaneously a thermodynamic cycle, a structural stress field, a rotating mechanical system, a fuel flow network, and a control system. These interactions are not peripheral — they are often where the most important system-level behaviors emerge.

9.1 Why Single-Domain Simulation Is Insufficient

The cross-domain interactions that co-simulation captures include phenomena that are invisible to any single-domain solver: heat generated by power electronics affects material properties, which affect structural behavior, which affects dynamic response, which affects control performance, which affects operating point, which determines heat generation. This circular causality is the signature of tightly integrated multi-domain systems.

For safety-critical systems, these interactions are also where failure modes emerge. Thermal runaway in a battery pack involves electrochemical, thermal, and electrical domain interactions. Flutter in an aircraft involves aerodynamic, structural, and control domain interactions. Co-simulation is the only computational approach that makes these failure modes accessible to analysis before physical testing.

9.2 The Multi-Domain Aircraft ECS Example

Environmental Control Systems in commercial aircraft illustrate multi-domain co-simulation at industrial scale. The ECS simultaneously involves:

- **Thermodynamics (Modelica/ThermoSysPro):** Bleed air extraction, expansion through turbines, mixing with recirculated cabin air, distribution through duct networks.
- **Structural mechanics (Nastran/ANSYS):** Fuselage pressure vessel stress under cabin-ambient pressure differential, thermal cycling fatigue from ground cold-soak to hot bleed air at cruise.
- **Controls (Simulink):** Pack flow control valve, mix manifold valve, and temperature control valve driven by sensor feedback through PID logic.
- **Propulsion offtake (NPSS/Modelica):** Bleed air extraction as a parasitic engine load affecting specific fuel consumption and thrust.

Co-simulation connects all four domains through FMI, with the master algorithm managing communication point timing and variable exchange.

9.3 Stability and Accuracy

The co-simulation master algorithm must actively manage several sources of numerical difficulty:

Algebraic loops: Circular dependencies where FMU A's output at time t depends on FMU B's output at time t , which depends on A's output. Requires iteration or an artificial small delay. The master must detect loops at assembly time.

Event handling: Discontinuous state changes — valve snap-opens, relay trips, fluid phase changes — require the master to detect the event time precisely, halt all subsystems, reinitialize consistently, and resume. Missing events produces silently wrong trajectories.

Step size adaptation: Good masters monitor exchange variable rates of change, shrinking the communication step during fast transients and expanding it during slow evolution. This requires subsystems to accept variable step requests — not all FMU implementations support this robustly.

10. Thermal Simulation Integration

Thermal simulation occupies a special position in the MBSE stack: it is the coupling domain. Heat generation depends on electrical dissipation, friction, and chemical reactions. Heat removal depends on fluid flow. The resulting temperature field affects structural stiffness, material properties, electrical resistance, and reaction rates. Thermal is the domain that receives energy from almost every other physics and returns boundary conditions to almost every other physics.

10.1 Tool Landscape

Tool Category	Examples	Appropriate Use
Lumped-parameter networks	Siemens NX Thermal, ESATAN-TMS, Modelica Thermal	System-level thermal networks; spacecraft; fast co-simulation through FMI
FEA thermal	ANSYS Mechanical, Abaqus, MSC Nastran	Spatial temperature gradients within components; junction temperatures; PCB heat spreading
CFD with conjugate HT	ANSYS Fluent, Siemens Star-CCM+, OpenFOAM	Convective cooling; electronics enclosures; heat exchangers; highest fidelity
1D thermal-fluid networks	Flownex, Flowmaster, Modelica Fluid	Cooling circuits, HVAC ducting, pipe networks; between lumped-parameter and full CFD

10.2 The Four MBSE Integration Points

Requirements layer: Thermal requirements in the SysML v2 model define simulation acceptance criteria as formal constraint expressions evaluable directly against simulation output. A RequirementUsage carrying "junction temperature $\leq 150^{\circ}\text{C}$ " is checkable without human comparison.

Interface definition layer: SysML internal block diagram or v2 port declarations define what thermal quantities (temperature, heat flow rate) cross subsystem boundaries. The co-simulation coupling configuration must implement exactly these interfaces — and a query against the SysML v2 API combined with FMU model description XMLs can verify it does.

Physics simulation layer: The thermal simulation runs, producing temperature fields, heat flux distributions, maximum temperatures, and thermal resistance maps as primary deliverables.

Cross-domain feedback layer: Temperature fields feed structural FEA as load cases. Thermal time constants feed control system plant models. Junction temperatures feed Arrhenius-based reliability analysis. Maximum temperature and cooling power feed Pareto optimization objectives.

10.3 Thermo-Mechanical Coupling

One-way (sequential) coupling: Run thermal analysis first, export nodal temperature field, import into structural model as body load, run stress analysis. Adequate when deformations do not

significantly change thermal boundary conditions. Computationally tractable and organizationally convenient.

Two-way (iterative) coupling: Required when deformations affect boundary conditions — turbine blade tip clearance changes that modify convective heat transfer, for example. Requires either tight co-simulation through FMI or a coupled solver within a single tool such as ANSYS coupled-field elements.

Mesh mapping: Thermal and structural meshes are optimized for different physics and are almost never identical. Conservative interpolation between them — preserving integrated heat content — is handled by tools such as ANSYS System Coupling or MpCCI. Non-conservative mapping produces artificial thermal stresses.

10.4 Reduced-Order Models for System Integration

A full conjugate heat transfer CFD model at 10 million cells taking 8 hours per steady-state point is incompatible with system-level co-simulation. Reduced-order models bridge this gap:

- **Compact thermal models:** Replace detailed components with resistance-capacitance networks reproducing external thermal behavior (junction-to-case resistance, thermal capacitance). The standard approach in electronics thermal simulation.
- **POD ROMs:** Proper Orthogonal Decomposition extracts dominant spatial modes from offline high-fidelity runs, representing new conditions as linear combinations solvable in milliseconds with errors typically below 1%.
- **Data-driven surrogates:** Gaussian Process regression or neural networks trained on simulation data. Fast to evaluate; no physical insight; failure outside training envelope is silent — requires careful validation.

11. Controls Simulation and Architecture Integration

Controls simulation and system architecture modeling have historically operated in parallel that only intersect at program reviews — the systems engineer draws the architecture, the controls engineer builds the Simulink model, and the connection is a document describing requirements that may or may not accurately reflect what either model implements. Closing this gap is one of the central value propositions of MBSE.

11.1 The Abstraction Mismatch

A system architecture model in SysML describes what the system is: components, interfaces, requirements, behavioral specifications at a functional level. A controls simulation in Simulink describes how a specific control law behaves when connected to a specific plant model under specific operating conditions. The architecture model is declarative and multi-stakeholder; the controls model is computational and specialist.

The gap is not accidental — it reflects a genuine difference in purpose. The challenge is building traceability and data flow connections that allow changes in one to propagate appropriately to the other, and allow verification evidence generated in controls simulation to close against requirements expressed in the architecture model.

11.2 The Plant Model Evolution

The plant model — the mathematical representation of the physical system being controlled — evolves in fidelity as design matures:

1. **Early phase:** Linearized analytical model (transfer functions, state-space matrices from first principles). Tractable for initial control law structure selection and gain calculation.
2. **Mid phase:** Modelica model of the physical subsystem validated against component test data, exported as FMU. The control law tuned against the simple plant may need retuning — or may reveal stability margins were inadequate when the simplified plant masked certain dynamics.
3. **Late phase:** Full co-simulation with the MBSE plant model including all cross-domain interactions. The control law is verified against the highest-fidelity available representation of the actual system.

The critical risk is that the plant model fidelity used for each requirement verification must be traceable. A control law designed against a linearized plant and one validated against a full nonlinear multi-physics FMU provide fundamentally different verification confidence levels.

11.3 Integration Mechanisms

Manual interface specification: Systems engineer defines the control interface in SysML; controls engineer implements as Simulink bus objects and validates conformance by inspection. Universal baseline; fragile — changes in SysML do not automatically propagate to Simulink.

Syndeia and bidirectional adapters: Tools that read SysML interface definitions, generate Simulink bus type definitions, and check existing Simulink models for conformance. Reduces drift between models without eliminating the need for human judgment about the meaning of changes.

SysML v2 API-driven integration: Python or Julia scripts query the SysML v2 repository for interface definitions and use that information to configure or validate the Simulink model. Scriptable, automatable, CI-pipeline compatible. Requires script maintenance.

11.4 Control Architecture Patterns

Mode logic: Supervisory state machines that select which control law is active belong in both the SysML architecture model and the Simulink Stateflow implementation. The architecture-level state machine specifies requirements; the Stateflow chart implements them. Keeping them synchronized is the primary integration maintenance challenge.

Gain scheduling: Scheduling variables (Mach number, altitude, battery SOC) are system-level quantities defined in the architecture model. The gain tables are control law implementation details. Interface mismatches — different units between the architecture's signal definition and the scheduler's input — are exactly what formally typed SysML v2 port declarations catch automatically.

Fault detection and reconfiguration: Fault modes declared in the architecture model (AADL EMV2, SysML failure mode allocations) must correspond to detection thresholds implemented in the controller. The traceability link between declared failure mode and implemented detection is a common source of drift that only surfaces as missed detections in late-phase testing.

11.5 The Verification Progression: MiL to HiL

Stage	Hardware	What It Verifies	What It Misses
MiL (Model-in-the-Loop)	None (all simulation)	Control law behavior against idealized plant	Code generation artifacts, fixed-point effects, timing
SiL (Software-in-the-Loop)	Compiled code on PC	Compiled code behavior; fixed-point arithmetic; quantization	Processor-specific timing; I/O electrical characteristics
PiL (Processor-in-the-Loop)	Real target processor	Real-time execution timing; interrupt latency; RTOS scheduling	I/O electrical characteristics; communication bus loading
HiL (Hardware-in-the-Loop)	Real ECU + real-time sim	Closed-loop system with all hardware effects	Full physical system integration; environmental stresses

12. Hardware-in-the-Loop Verification

Hardware-in-the-loop (HiL) simulation replaces part of a real system with a real-time simulation while leaving the rest as actual physical hardware — creating a closed-loop test environment where real hardware believes it is operating in its actual system context, even though most of that context is computed. The word "real-time" carries precise meaning: the simulation must complete each computational cycle within the same wall-clock time as the physical system's control cycle — typically 1ms to 10ms for automotive and aerospace applications, sometimes microseconds for power electronics.

12.1 Physical Architecture of a HiL Rig

Real-time target: A dedicated computer with a deterministic real-time operating system — dSPACE SCALEXIO, NI PXI, or Speedgoat Performance target. Determinism means the simulation step executes in exactly the same wall-clock time every cycle, with no jitter from OS scheduling or background tasks. This determinism is what allows the hardware under test to trust the signals it receives.

I/O interface: A rack of signal conditioning hardware: D/A converters generating analog sensor voltages, A/D converters reading actuator command voltages, digital I/O, CAN/ARINC 429/664 bus interfaces, PWM generators, and custom interface hardware. The electrical characteristics must match what the unit under test expects — voltage ranges, impedance, rise times, noise levels.

Unit under test (UUT): The real hardware — ECU, flight control computer, BMS, or domain controller — running actual production software in its actual enclosure. It does not know it is connected to a simulation.

Plant simulation: A real-time mathematical model of everything the UUT is not — vehicle dynamics, aircraft aerodynamics, battery chemistry, hydraulic actuators, thermal environment. This runs on the real-time target and is assembled from FMUs for maximum fidelity and reusability.

12.2 What HiL Uniquely Reveals

HiL testing surfaces failure modes that no earlier simulation stage can detect:

- **Real-time execution effects:** A control loop designed for 10ms execution may actually take 10.3ms on average with occasional 12ms spikes due to cache effects and branch misprediction. These timing violations introduce phase lag that degrades stability margins below what SiL testing showed.
- **I/O hardware characteristics:** Real sensors have output impedance, capacitance, noise spectral density, and temperature-dependent offset drift that differ from simulation models. Real actuator drivers have switching transients and current limiting behaviors affecting the control loop.
- **Communication bus loading:** CAN bus arbitration contention under realistic multi-node loading causes message delivery timing variations that SiL testing with perfect message delivery does not exercise.
- **Failure mode injection:** Sensor drift, actuator failures, power supply voltage sags, communication bus message loss — all can be injected controllably and repeatedly, enabling systematic verification of fault detection and reconfiguration logic before the first physical system integration.

12.3 HiL in the Certification Evidence Chain

DO-178C (avionics): HiL testing contributes to requirements-based testing at the system level. Test cases are derived from SysML architecture requirements, executed against production software on real hardware in a realistic plant, and results documented as traceable verification evidence.

ISO 26262 (automotive): ASIL D functions — the highest safety level — require comprehensive HiL testing with documented fault injection results. The standard distinguishes software-level verification (MiL, SiL) from system-level validation (HiL, vehicle tests).

In both domains, the certification authority reviews not just test results but test coverage rationale: how do the selected test cases provide evidence of requirement satisfaction under all relevant operating conditions? This is a systems engineering activity requiring reasoning about the relationship between test coverage and requirement satisfaction — precisely what the MBSE architecture model supports.

12.4 HiL in the MBSE Knowledge Graph

A mature program may run thousands of HiL test cases across multiple plant simulation configurations, software versions, and hardware revisions. In a knowledge graph architecture, each test case is a node connected by typed edges to the SysML v2 requirement it exercises (via REST API), the plant model configuration (FMU versions and parameter values), the software build under test, and the test result with timestamp and operator identity.

A query against this graph can answer: "Which requirements have no HiL test case that has passed in the last six months against the current software build?" — the configuration management query that catches verification gaps before a certification audit.

The Circuit Breaker principle: before a HiL test configuration executes, validate it against the SysML v2 interface declarations in the repository. A test that injects a temperature signal in Fahrenheit to a controller expecting Celsius will look like a pass — it tests the wrong behavior. Grounding the test configuration against formally typed architectural declarations catches this before execution, not during post-test analysis.

PART IV

Integration Architecture and AI Grounding

13. The Integrated MBSE Architecture

The methods, languages, and tools described in this document are most powerful when used in concert. The integrated architecture connects them through a federated knowledge graph that makes cross-tool traceability a first-class, queryable capability rather than a manually maintained document.

13.1 The Complete Integration Pipeline

In a mature program, the five-layer pipeline operates as follows:

4. **SysML v2 (architecture)**: Captures system requirements, functional allocation, interface definitions with typed quantity kinds and units. Stored in a REST API repository accessible to all downstream tools.
5. **Modelica / AADL (physics and SW architecture)**: Instantiates subsystem physics and software timing behavior. Validated against component test data. Exported as FMUs for system-level integration.
6. **FMI co-simulation (integration)**: Assembles FMUs from multiple tools into an integrated virtual system. The co-simulation topology is derived from the SysML v2 interface declarations — automatically or with minimal manual configuration.
7. **RSM surrogates (optimization enablement)**: DoE-samples the co-simulation at strategically chosen operating points, fitting polynomial or Kriging surrogates for each system-level objective. Makes the design space explorable without full simulation cost.
8. **Pareto analysis (decision support)**: Multi-objective optimization sweeps the surrogates to generate the Pareto frontier. Stakeholders select from the frontier with full awareness of the trade-offs, grounded in the high-fidelity simulation that underlies the surrogate.

13.2 The Knowledge Graph as Traceability Backbone

The SysML v2 model repository, FMU model description XMLs, AADL component specifications, RSM surrogate parameters, Pareto frontier records, and HiL test results are nodes in the same knowledge graph with traversable, typed edges between them. This enables cross-domain queries that are impossible with siloed tools:

- Which system-level performance objectives are currently supported by a HiL-validated surrogate, and which rely on MiL-only simulation?
- For any requirement that was recently modified in the SysML v2 repository, what simulation analyses and HiL tests depend on the previous version and now need re-execution?
- What is the provenance chain from the current Pareto frontier to the simulation configurations, mesh files, boundary conditions, and software versions that produced the objective function values?
- Which FMU interface variables have units declared in the model description XML that do not match the quantity kind declared in the SysML v2 port definition?

13.3 The Circuit Breaker Pattern for AI Grounding

As AI assistants are increasingly deployed to support engineering workflows — recommending simulation configurations, interpreting results, generating model fragments — the risk of AI-generated assertions that are plausible but incorrect becomes a system safety concern. The Circuit Breaker pattern addresses this by grounding AI outputs against the authoritative engineering knowledge graph before they are presented to engineers.

The mechanism: Before an AI assistant makes a claim about system architecture, interface specifications, or simulation configurations, it retrieves the relevant model elements from the SysML v2 repository, FMU model descriptions, or verification evidence store and checks its assertion against the retrieved ground truth. Discrepancies trigger a hold rather than silent propagation.

Mean Time Between Hallucinations (MTBH): A proposed metric for AI grounding effectiveness in engineering contexts — the rate at which AI-generated engineering assertions that were not validated against the knowledge graph later prove inconsistent with verified system specifications. Lower MTBH indicates better grounding discipline. The metric is measurable because the knowledge graph provides an auditable ground truth against which AI assertions can be retrospectively verified.

The structural parallel: model drift in physics simulation (a Modelica model that has diverged from calibrated physical reality) and AI hallucination in language models (a confident assertion that has diverged from ground truth) are the same failure mode at different abstraction levels. Both are addressed by the same architectural remedy: a live, queryable, version-aware connection between the generating system and an authoritative source of verified truth.

14. Conclusions and Practitioner Guidance

The transition from document-based to model-based systems engineering is not primarily a technology adoption challenge — it is an information architecture challenge. The tools exist. The standards exist. What programs struggle with is the semantic integration layer: making the connections between architectural intent, physics simulation, verification evidence, and stakeholder requirements first-class, queryable artifacts rather than implicit knowledge maintained in documents.

14.1 Method Selection Guidance

If your primary question is...	Use this method	With this tooling
Where is the optimum in my parameter space?	RSM	Design-Expert, JMP, Python/scikit-learn
What trade-offs are unavoidable between my objectives?	Pareto analysis + NSGA-II	MATLAB gamultiobj, pymoo, Platypus
What does my integrated system actually do?	MBSE co-simulation	Modelica + Simulink + ANSYS via FMI
Will my software meet real-time deadlines on target hardware?	AADL + schedulability analysis	OSATE, TASTE
Does my controller meet stability requirements on real HW?	HiL testing	dSPACE, NI VeriStand, Speedgoat
Are my simulation results traceable to requirements?	Knowledge graph integration	SysML v2 API + Neo4j or RDF store

14.2 Common Integration Failure Modes

Programs that invest in individual MBSE tools without the integration architecture consistently encounter the same failure modes:

- **Model drift:** The SysML architecture model and the Simulink simulation model diverge over time because changes in one are not propagated to the other. Interface mismatches accumulate silently and manifest as incorrect simulation results or test failures late in the program.
- **Verification coverage gaps:** Requirements added late in the program have no corresponding simulation analyses or HiL test cases. The gap is only discovered during a certification audit.
- **Surrogate scope violations:** RSM surrogates built against a simplified or outdated plant model are used to support design decisions that depend on dynamics the surrogate does not capture.
- **Unit and quantity mismatch:** An interface variable declared as a mass in kilograms in the architecture is connected to a block that expects pounds in the simulation. The simulation runs

without error because the software types match; the results are wrong because the physical quantities do not.

- **Provenance loss:** Pareto frontier results cannot be traced back to the specific simulation configurations, software versions, and boundary conditions that produced them. The frontier cannot be reproduced or extended as design matures.

14.3 The Path Forward

The most significant near-term opportunity in MBSE practice is the SysML v2 REST API ecosystem. As tool implementations mature through 2025-2027, the architectural possibility of a federated, tool-neutral model repository — connected to simulation tools through FMI, to requirements management through direct API integration, and to knowledge graphs through ETL pipelines — moves from specification to practice.

Programs that invest now in the integration architecture — even before all tools support SysML v2 natively — are building the traceability foundations that will enable model-based certification evidence, AI-assisted simulation setup, and continuous verification as requirements evolve. Programs that wait for tool maturity will find themselves building the integration architecture under program schedule pressure, which is when integration shortcuts are taken and architectural debt accumulates.

The three methods examined in Part I — RSM, Pareto analysis, and MBSE simulation — are not competitors. They are collaborators in a pipeline where each addresses the part of the engineering challenge the others cannot. Building the infrastructure that connects them is the work that separates programs that extract the full value of MBSE from programs that have impressive tools running in productive isolation.

The central recommendation: treat the connections between tools as first-class engineering artifacts that require the same rigor, versioning, and maintenance as the models themselves. Interface definitions, unit specifications, FMU coupling configurations, verification evidence traceability, and provenance chains are not administrative overhead — they are the semantic foundation on which model-based engineering's value proposition rests.

Appendix A: Quick Reference — Key Acronyms and Terms

Term	Full Name	Context
AADL	Architecture Analysis and Design Language	SAE AS5506; safety-critical embedded SW/HW architecture
CFD	Computational Fluid Dynamics	Numerical simulation of fluid flow and heat transfer
CCD	Central Composite Design	RSM experimental design with factorial + axial + center points
CHT	Conjugate Heat Transfer	Simultaneous solid and fluid thermal simulation
DAE	Differential-Algebraic Equation	Equation system type solved by Modelica; mixes ODEs and constraints
DoE	Design of Experiments	Systematic selection of experimental or simulation run configurations
ECS	Environmental Control System	Aircraft cabin pressurization and temperature management
EMO	Evolutionary Multi-objective Optimization	Algorithm class for generating Pareto frontiers (NSGA-II, NSGA-III)
FEA	Finite Element Analysis	Numerical structural and thermal analysis on discretized mesh
FMI	Functional Mockup Interface	OMG standard for simulation tool interoperability
FMU	Functional Mockup Unit	Self-contained simulation component conforming to FMI specification
HiL	Hardware-in-the-Loop	Real hardware connected to real-time plant simulation
HTC	Heat Transfer Coefficient	Convective thermal boundary condition parameter
KerML	Kernel Modeling Language	Semantic foundation of SysML v2; replaces UML profile approach
LRU	Line Replaceable Unit	Aircraft avionics unit replaceable at the flight line level
MBSE	Model-Based Systems Engineering	Use of formal models as primary artifacts across the lifecycle

MiL	Model-in-the-Loop	Control algorithm tested against simulation plant with no real hardware
MSL	Modelica Standard Library	Standard component library shipping with all Modelica tools
MTBH	Mean Time Between Hallucinations	Proposed AI grounding metric for engineering applications
NSGA-II	Non-dominated Sorting Genetic Algorithm II	Dominant EMO algorithm for 2-3 objective Pareto optimization
PiL	Processor-in-the-Loop	Compiled code running on real target processor, simulation plant on PC
POD	Proper Orthogonal Decomposition	Method for extracting spatial ROM basis from high-fidelity simulations
RMA	Rate Monotonic Analysis	Formal schedulability proof method for fixed-priority real-time systems
ROM	Reduced-Order Model	Low-dimensional approximation of high-fidelity simulation for FMI export
RSM	Response Surface Methods	Statistical surrogate modeling approach for design optimization
SiL	Software-in-the-Loop	Compiled production code tested in simulation on PC
SysML	Systems Modeling Language	OMG standard architectural modeling language for MBSE
WCET	Worst-Case Execution Time	Maximum time a software task requires; declared in AADL thread properties

Three approaches

These three approaches represent distinct schools of thought in decision-making under complexity. Here's how they compare across several key dimensions, followed by a visual map.

Purpose and philosophical grounding

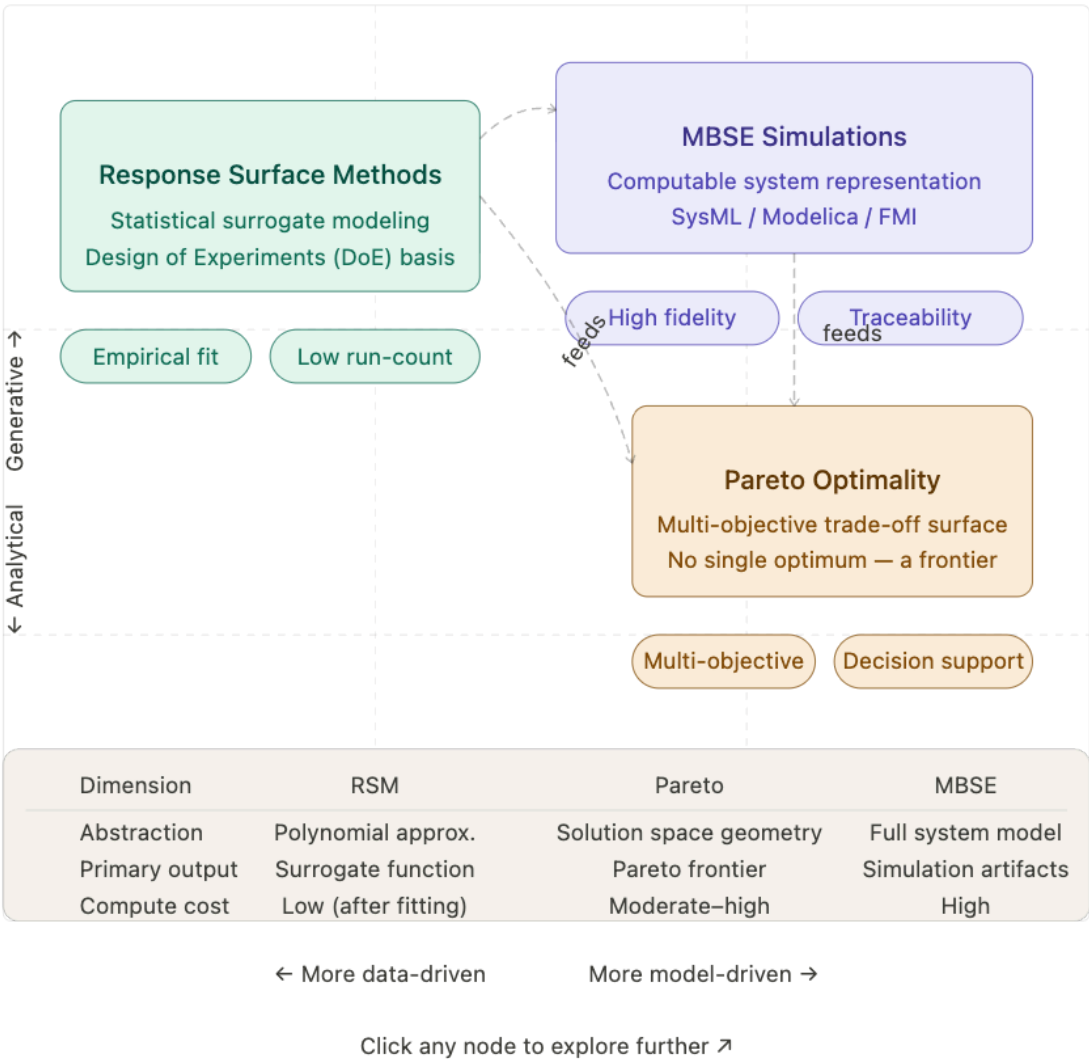
Pareto Optimality Analysis is fundamentally about *trade-off characterization* — it doesn't find "the answer," it finds the set of solutions where you cannot improve one objective without degrading another. It tells you what's possible, not what to choose. Response Surface Methods (RSM) are about *empirical landscape mapping* — fitting a polynomial approximation to a complex system's behavior so you can navigate it efficiently without running exhaustive experiments. MBSE simulations pursue *federated fidelity* — building computable, semantically rich representations of a system that can be exercised, verified, and evolved alongside the real system.

What each optimizes for

RSM optimizes for statistical efficiency in exploration. Pareto analysis optimizes for transparency of trade-off space. MBSE simulations optimize for fidelity and traceability. Crucially, RSM and MBSE can *feed into* Pareto analysis — they generate the data or model outputs that a Pareto frontier is drawn from.

Maturity and integration

These methods exist on a spectrum from data-centric to model-centric. RSM is fundamentally a statistical curve-fitting technique — it approximates. MBSE is architecturally structural — it *represents*. Pareto analysis is an evaluative lens that sits atop both. In sophisticated systems engineering contexts (aerospace, pharma, defense), all three are often used in concert: MBSE provides the structural model, RSM generates a surrogate model of expensive simulations, and Pareto surfaces communicate trade-offs to decision-makers.



Key contrasts in practical use

Scope of validity. RSM is valid only within the design space where experiments were run — extrapolate outside that region and the polynomial surface becomes unreliable. MBSE simulations, built from physics or logic principles, can be exercised in scenarios that were never explicitly tested. Pareto analysis has no inherent scope limitation; it reflects whatever objective functions you define.

What they assume about the system. RSM assumes the system behaves smoothly enough that a low-order polynomial can approximate it — this fails badly for highly nonlinear or discontinuous systems. MBSE assumes you can formalize system behavior in a computable model (SysML, Modelica, FMI, etc.) — a significant upfront investment. Pareto analysis makes no assumptions about the system itself; it just requires that you can evaluate competing objectives.

Who they serve. RSM is primarily an engineer's tool — it compresses experimentation cost. MBSE is an architect's and systems engineer's tool — it creates a living specification that bridges requirements, design, and verification (which aligns closely with your work at INCOSE and in the DoD LVC work). Pareto analysis is a decision-maker's tool — it structures the conversation between engineers and stakeholders about what trade-offs are acceptable.

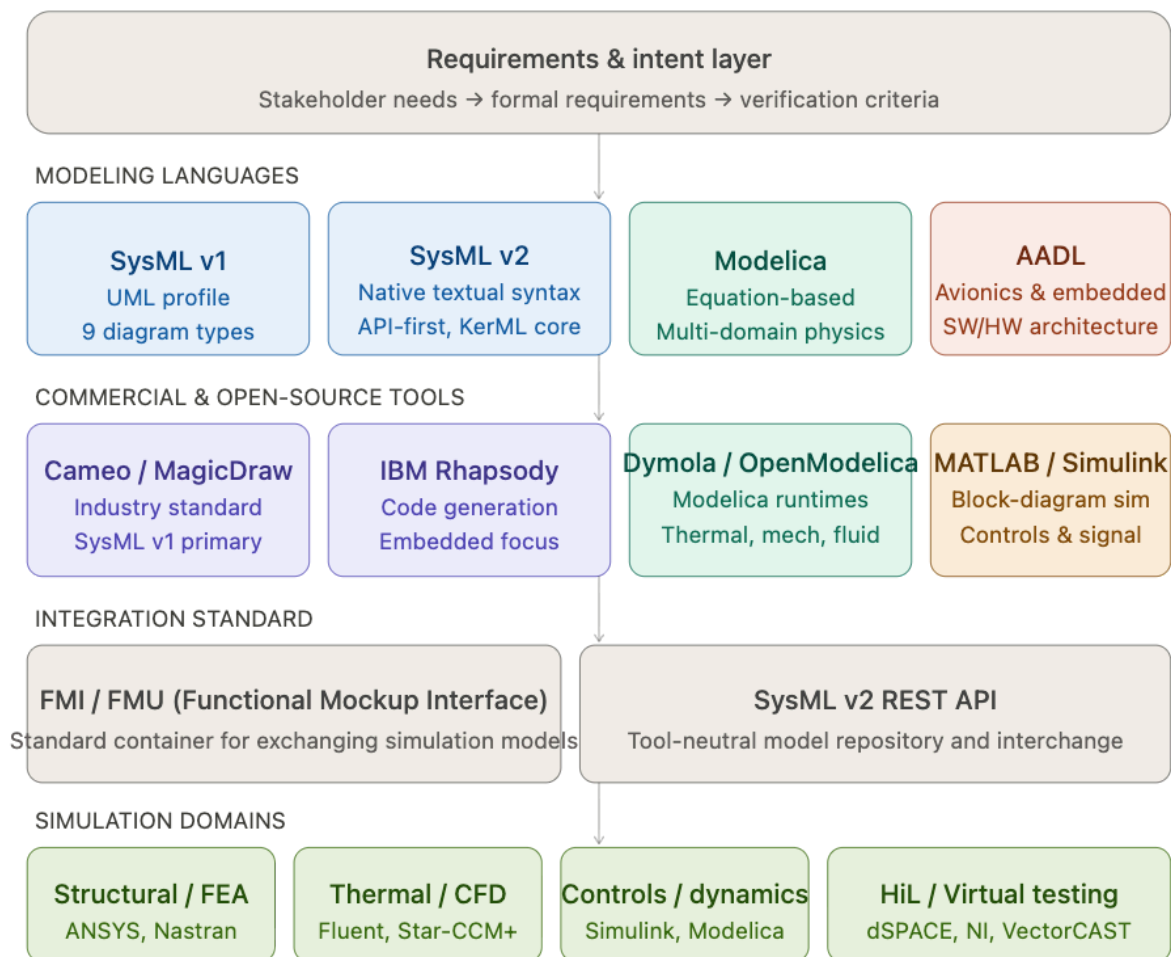
Where they break down. RSM breaks when you have too many input factors (the curse of dimensionality makes the DoE matrix explode) or when the response surface is multimodal. MBSE breaks when model fidelity is aspirational rather than validated — a model that doesn't reflect ground truth gives precise answers to the wrong questions, a failure mode you've written about in the context of AI grounding. Pareto analysis breaks when objectives are poorly defined or when decision-makers refuse to engage with the trade-offs — producing a beautiful frontier that gets ignored.

Integration pattern in mature systems engineering

The most powerful usage pattern combines all three: MBSE provides the high-fidelity simulation backbone; RSM builds a cheap surrogate model by sampling that simulation strategically (DoE); the surrogate is then swept across a multi-objective optimization to generate a Pareto frontier; and stakeholders use the frontier to make explicit trade-off decisions. This is standard practice in aerospace MDO (multidisciplinary design optimization) and increasingly in pharmaceutical process development — both domains you've worked in deeply.

MBSE (Model-Based Systems Engineering)

MBSE (Model-Based Systems Engineering) simulations rest on a core idea: replace documents as the primary system artifact with a *computable, semantically rich model* that can be queried, verified, simulated, and evolved. The model is the truth — not the Word doc that describes it.



Click any node to explore further ↗

Modeling languages — the semantic backbone

SysML v1 is the workhorse of the past two decades. It's a UML profile — nine diagram types covering requirements (req diagrams), structure (BDD, IBD), behavior (activity, sequence, state machine), and parametrics (which is where it connects to simulation by expressing constraint equations across system properties). The parametric diagram is SysML's bridge to math — you define that thrust equals mass times acceleration, and downstream tools can solve that constraint.

SysML v2 is a ground-up redesign released as a standard in 2024. It abandons the UML profile approach, introduces a native textual syntax (KerML as the foundation, SysML v2 on top), and most

importantly ships with a REST API — meaning models live in a repository that any conformant tool can query or write to. This is architecturally transformative: it shifts MBSE from "one vendor's proprietary model file" to "shared semantic graph accessible programmatically." The resonance with your ARK/Neo4j work at AbbVie should be immediate — it's the same federated knowledge graph philosophy applied to engineering artifacts.

Modelica operates at a different abstraction level. Rather than describing architecture, it expresses multi-domain physical systems as differential-algebraic equations. A *Modelica* model of a thermal management system isn't a box-and-line diagram — it's equations for heat transfer, fluid dynamics, and electrical behavior that a solver integrates over time. It's the language of physics simulation, not system description.

AADL (Architecture Analysis and Design Language) is the embedded/avionics specialist — strong in software component architectures, timing analysis, and safety-critical systems where you need schedulability proofs and fault tree integration.

The toolchain reality

Cameo Systems Modeler (formerly MagicDraw + the SysML plugin, now part of Dassault's No Magic portfolio) is the de facto industry standard for SysML v1 work in aerospace and defense — it's what Gulfstream, Lockheed, Raytheon, and NASA programs typically mandate. IBM Rhapsody competes in the embedded/automotive space where it adds automatic C/C++ code generation from state machines — the model becomes the implementation, not just the specification.

On the physics simulation side, Dymola is the commercial *Modelica* runtime of choice for automotive and aerospace, while OpenModelica is the open-source alternative with growing adoption. MATLAB/Simulink occupies a unique position — it's not strictly an MBSE tool, but it's so deeply embedded in controls and signal processing workflows that most serious MBSE deployments treat it as a co-simulation peer, connected through FMI export.

FMI — the interoperability linchpin

The Functional Mockup Interface standard is what makes multi-domain co-simulation practical. A tool like Dymola or Simulink can export a *Functional Mockup Unit* (FMU) — a self-contained black-box package with inputs, outputs, and an internal solver. An orchestration platform (Modelon Impact, Siemens Simcenter, or an open-source master like PyFMI) can compose multiple FMUs from different tools and step them forward in time together. This is how you simulate a rocket where aerodynamics runs in ANSYS Fluent, structural dynamics runs in Nastran, controls run in Simulink, and thermal behavior runs in a *Modelica* model — they exchange state at each time step without any tool needing to understand another's internals.

Where MBSE simulation fits in the V-model

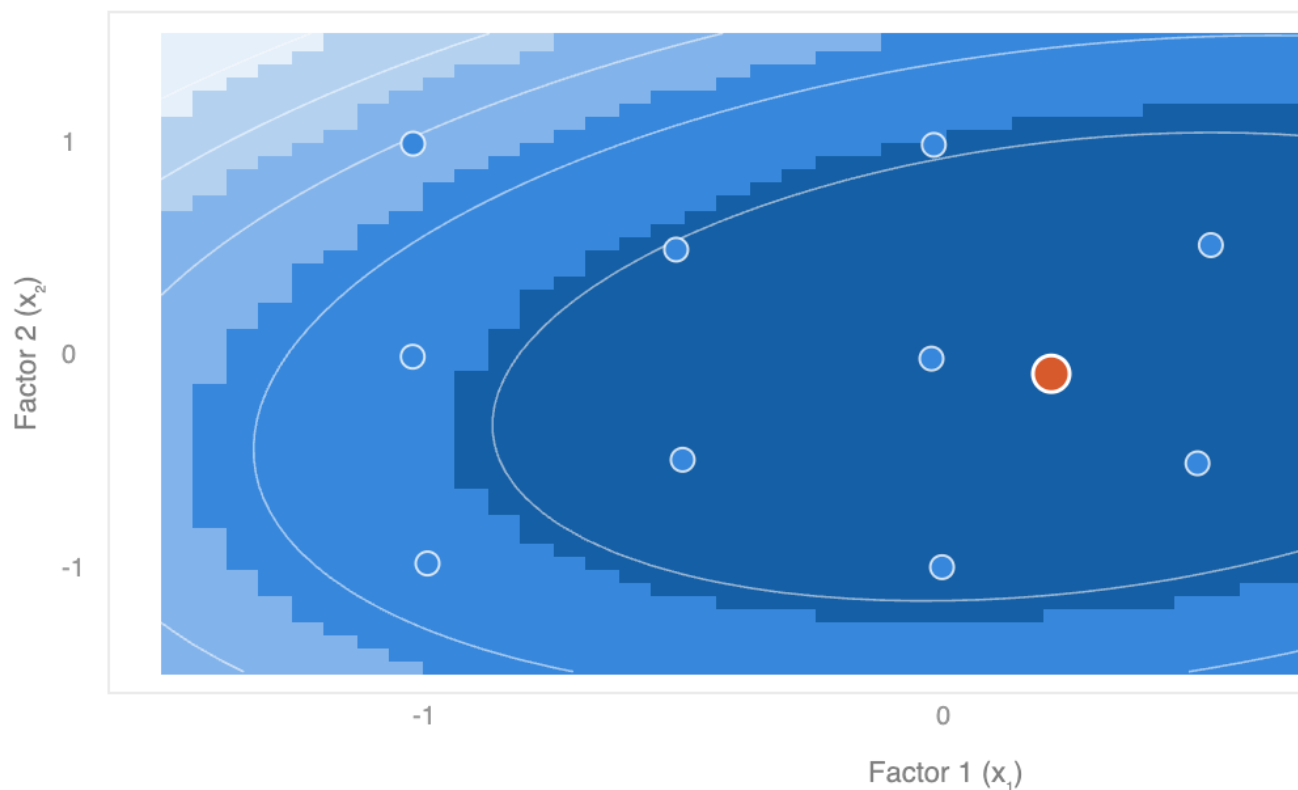
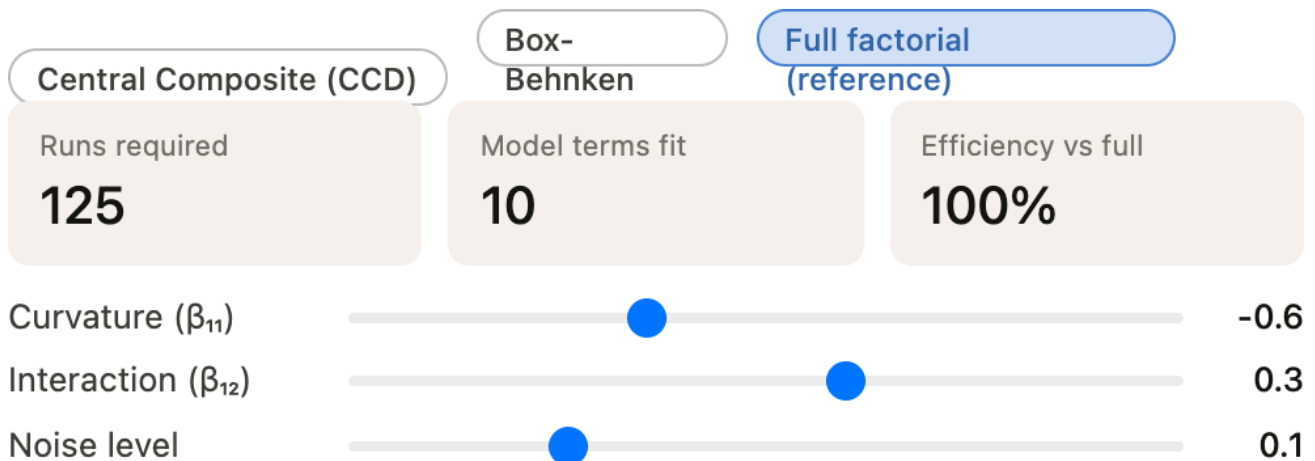
The architectural model in SysML/Cameo sits at the top of the V — capturing requirements, system decomposition, and interface definitions. As you descend the left side, *Modelica* or Simulink models

instantiate the physics of subsystems. FMI connects them for integrated vehicle-level simulation. Verification (right side of the V) closes the loop — simulation outputs are compared against the requirements captured in the same model, so traceability is structural, not documentary.

The honest limitations

Model fidelity is always aspirational until validated. The MBSE simulation stack is only as trustworthy as its calibration against physical test data — a beautifully structured SysML model with Modelica subsystems that have never been corroborated against hardware gives you precise answers to potentially wrong questions. This is exactly the AI grounding problem you've framed with Circuit Breaker and MTBH — the mechanism of hallucination in an LLM and the mechanism of model drift in a physics simulation are structurally identical: a system confidently producing outputs that have drifted from the ground truth it purports to represent.

The toolchain fragmentation is also real. SysML v2's REST API is the most credible attempt yet to break vendor lock-in, but as of now most production programs still run SysML v1 in Cameo with custom scripting to export data into PLM systems (Teamcenter, Windchill). The dream of the fully federated, tool-neutral MBSE repository remains partially aspirational — though your ARCH work at AbbVie is arguably ahead of most engineering programs in actually implementing that pattern.



● Design points ● Predicted optimum Contour lines = equal response

The widget above lets you feel the core trade-offs. Drag the curvature slider to see how a strongly curved surface (a sharp optimum) looks very different from a near-flat one — and notice that you need at least a quadratic model to detect it. Switch between the three designs to see how CCD and Box-Behnken place their points very differently while achieving similar efficiency compared to full factorial.

The mathematical machinery

The surface you're fitting is a second-order polynomial — for two factors it looks like this:

$$\hat{y} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_{12} X_1 X_2 + \beta_{11} X_1^2 + \beta_{22} X_2^2$$

The six terms capture: an intercept, two linear effects, one two-way interaction, and two pure quadratic (curvature) terms. With three factors you get 10 terms; with four factors, 15. This is why the curse of dimensionality bites RSM — the number of terms (and therefore the minimum run count) grows as $(k+1)(k+2)/2$ where k is the number of factors. Beyond 6–7 factors, you're typically better off using other screening strategies first.

The β coefficients are estimated by ordinary least squares once you have your experimental data. The DoE design determines the matrix of factor settings, and a good design ensures that matrix is well-conditioned — meaning small estimation errors and good predictive power in the region of interest.

Design of Experiments strategies within RSM

The Central Composite Design (CCD) is the workhorse. It combines a factorial core (the corners of the design space), center points (multiple runs at the center, which estimates pure error and curvature), and axial or "star" points that extend slightly beyond the factorial region along each factor axis. The star points are what give CCD its ability to fit the quadratic terms efficiently. CCD typically needs $2k + 2k + \text{center points runs}$ — for three factors, that's 8 factorial + 6 axial + 6 center = 20 runs.

Box-Behnken designs avoid the extreme corner points and axial points entirely, placing all runs at the midpoints of the design cube's edges plus center points. This is attractive when the corners represent physically dangerous or practically impossible combinations — think extreme temperature combined with extreme pressure combined with maximum catalyst loading. For three factors it needs only 13 runs versus CCD's 20, though the trade is slightly lower predictive efficiency at the edges.

When RSM is the right tool

RSM earns its place when three conditions align: the response is smooth and continuous (a polynomial can approximate it reasonably), you have a bounded, pre-screened set of factors (you're not still hunting which variables matter — that's a different problem for screening designs like Plackett-Burman or fractional factorials), and each run is expensive enough that the savings from a designed experiment over full factorial or trial-and-error are meaningful.

In your practice this maps directly to pharmaceutical process development — tablet coating and granulation parameters, reaction conditions in API synthesis, bioreactor optimization, lyophilization cycle development. ICH Q8/Q9/Q10 guidance on Quality by Design explicitly embraces RSM as a tool for establishing the design space — the region within which process parameters can move without negatively affecting product quality. The Pareto frontier then enters when you have multiple

quality attributes in tension (high dissolution rate but also low friability, for example) and need to characterize the trade-off surface rather than optimize a single response.

In aerospace and defense, RSM appears in aerodynamic shape optimization (ANSYS or SU2 runs are expensive, so you sample the design space, fit a Kriging surrogate — a more sophisticated version of the polynomial surface — and optimize on the surrogate), propulsion parameter studies, and structural weight-vs-strength trade-offs feeding into the Pareto analysis we discussed earlier.

Where RSM breaks

RSM assumes the surface is smooth enough for a second-order polynomial to capture it within the region of interest. If your system has discontinuities (phase transitions, yield drop-offs, controller saturation), the fitted surface will be badly wrong near those discontinuities and the predicted optimum may lie precisely at one of them. It also assumes stationarity — that the relationship between inputs and outputs doesn't drift over time or depend on factors you haven't included. In practice this means RSM is a snapshot, not a living model, which is one reason it feeds into the broader MBSE paradigm rather than replacing it: the RSM surrogate gives you a cheap local navigator, while the MBSE simulation gives you the high-fidelity ground truth you validate against.

The dimensionality ceiling is also real. Once you're above 5–6 factors in RSM, the run counts become prohibitive and you should consider machine-learning surrogates (Gaussian Process regression / Kriging, neural network metamodels) that can handle higher dimensions more gracefully — though they sacrifice the interpretability of knowing exactly what your β coefficients mean.

Pareto optimality is one of those ideas that sounds abstract until the moment it clicks — and then you see it everywhere. The fundamental insight is that in any multi-objective problem, there is no single "best" solution. Instead there is a *set* of solutions that are each unbeatable in their own way: you cannot improve any one objective without making at least one other objective worse. That set is the Pareto frontier.

Let me build this up from the geometry first, then connect it to construction methods and interpretation.

The geometry of dominance

The key concept is *Pareto dominance*. Solution A dominates solution B if A is at least as good as B on every objective, and strictly better on at least one. A solution is Pareto-optimal if nothing dominates it. The frontier is the complete set of non-dominated solutions.

Total solutions
30

On frontier
4

Dominated
26

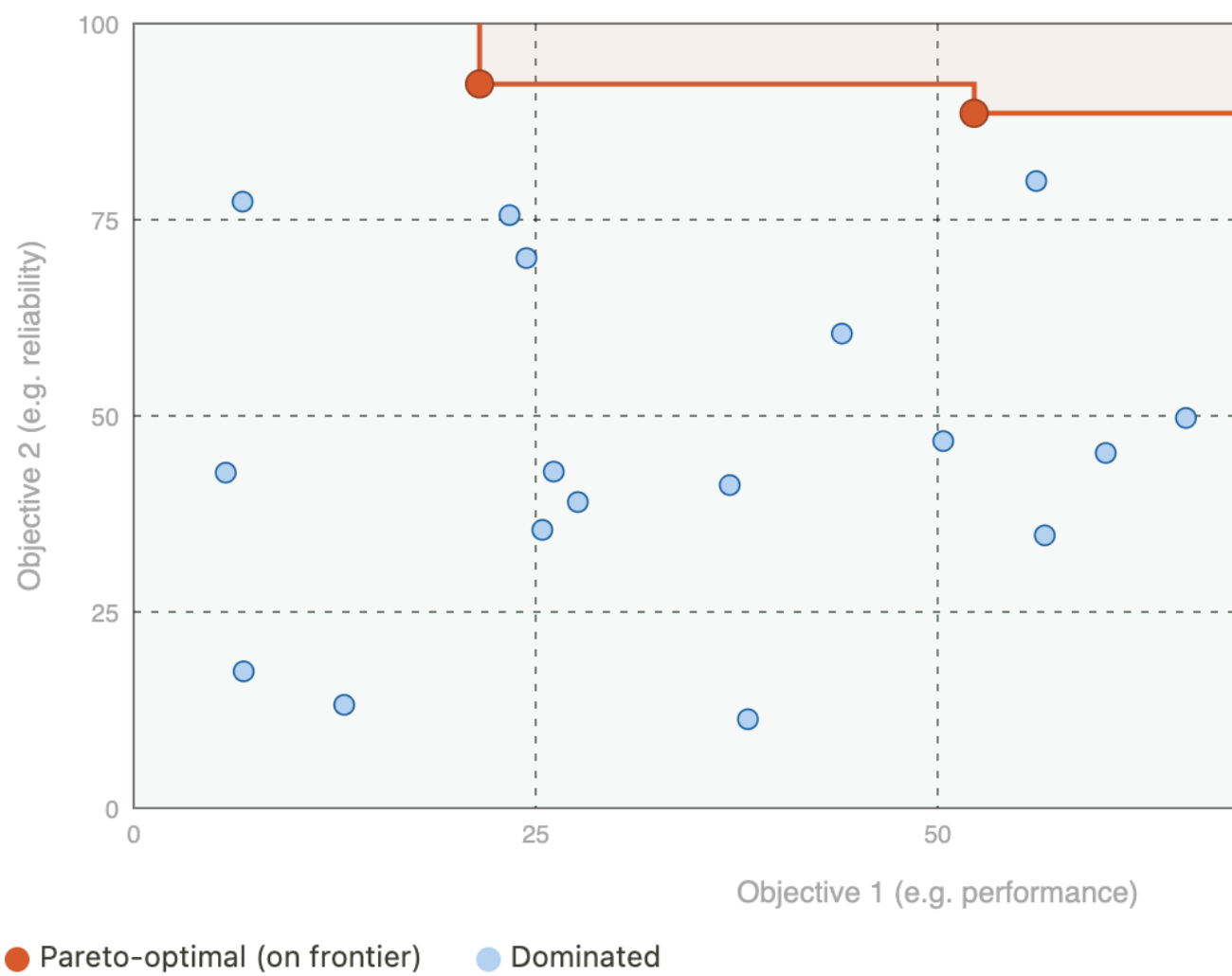
Add 20 random ↗

Add 50 random ↗

Clear

Show dominance zones

Click anywhere on the chart to add a solution. Hover a point to see what it dominates.



Both objectives: higher = better (maximize)

Click anywhere to add solutions, hover points to see their dominance relationships, and toggle the dominance zone overlay to see the region a Pareto-optimal point controls. Notice that as you add more solutions, the frontier gets pushed outward — it is always convex-ish in shape, and it always represents the best achievable trade-off given the population you have.

How the frontier is constructed

For a discrete set of candidate solutions (as above), construction is conceptually simple: test every pair for dominance, and retain only the non-dominated solutions. The naive algorithm is $O(n^2 \cdot k)$ where n is the number of solutions and k is the number of objectives — manageable for hundreds of solutions, expensive for millions.

For continuous problems — where you're searching an infinite design space — construction requires optimization algorithms specifically designed for multi-objective problems. The dominant family is evolutionary multi-objective optimization (EMO). The canonical algorithms are NSGA-II (Non-dominated Sorting Genetic Algorithm II) and its successor NSGA-III for three or more objectives. These work by maintaining a *population* of candidate solutions, selecting parents based on Pareto rank and crowding distance (a measure of spacing along the frontier — you want to preserve diversity, not cluster all solutions in one region), and generating offspring through crossover and mutation. Over many generations the population converges toward the true Pareto frontier.

When the objectives are computed by expensive simulations — CFD runs, FEA models, MBSE simulation outputs — you typically can't afford thousands of function evaluations. This is where RSM enters as the natural partner: you build surrogate models of each objective using DoE and RSM, then run the NSGA-II optimization on the cheap surrogates. The final frontier candidates are then verified against the high-fidelity simulation.

The shape of the frontier and what it tells you

The frontier's curvature is informative. A nearly linear frontier between two objectives means the trade-off is approximately constant — sacrificing one unit of objective A always buys roughly the same gain in objective B, regardless of where you are on the frontier. A strongly curved frontier means the trade-off is asymmetric — there may be a "knee point" where you get excellent performance on both objectives simultaneously, and beyond which you give up a lot of one to gain a little of the other. The knee is typically where engineers focus attention because it represents maximum efficiency in the trade-off.

A disconnected or fragmented frontier — gaps in the solution set — signals a multimodal landscape with distinct design paradigms that don't blend smoothly. In aerospace, a fragmented weight-vs-cost frontier might reflect two fundamentally different structural concepts (composite vs aluminum) where intermediate hybrid designs are actually worse than either pure approach.

Hypervolume as a scalar quality metric

When you're comparing two optimization runs, or tracking convergence, you need a single number

to characterize frontier quality. The standard metric is *hypervolume indicator* — the volume of objective space dominated by the frontier relative to a reference point (typically the worst achievable values on each objective). A frontier with larger hypervolume is better in a Pareto sense: it either has solutions closer to the ideal, or they're more spread out, or both. This is how algorithm developers benchmark NSGA-II against competitors — not by counting solutions, but by measuring the dominated volume.

From frontier to decision

This is where Pareto analysis becomes a governance tool rather than a mathematical exercise — and where it connects most directly to your semantic governance and stakeholder alignment work. The frontier doesn't make the decision. It structures the decision conversation by making trade-offs explicit and removing the pretense that there is an objectively "correct" answer.

The standard approaches for selecting from the frontier are worth naming:

Weighted sum scalarization is the simplest: assign weights to each objective reflecting stakeholder priorities, compute a weighted score for each frontier point, and select the maximum. The weakness is that weights must be specified *a priori*, and decision-makers often cannot reliably articulate weights before they see the frontier — they discover their preferences by looking at the trade-offs. Also, weighted sums cannot reach non-convex regions of the frontier.

Reference point methods (used in NSGA-III and the epsilon-constraint method) let the decision-maker specify aspiration levels — "I need at least 85% reliability and at most \$2M cost" — and the algorithm finds frontier points closest to that reference. This is more natural for engineering stakeholders who think in terms of requirements rather than abstract weights.

Knee point selection is algorithmic: identify the frontier point with maximum curvature (or maximum distance from the line connecting the frontier's endpoints). This is often the engineer's default when stakeholders cannot articulate preferences — it's the point that extracts maximum simultaneous value.

The three-objective and beyond problem

With two objectives, the frontier is a curve you can draw and show. With three objectives it becomes a surface — still visualizable with 3D scatter plots, but harder to communicate. Beyond three objectives — which is common in real systems engineering (weight, cost, reliability, performance, manufacturability, maintainability...) — the frontier becomes a high-dimensional manifold that is genuinely difficult to interpret. This is the domain of *multi-criteria decision analysis* (MCDA) methods like TOPSIS, VIKOR, and outranking methods (ELECTRE, PROMETHEE) that try to impose structure on the decision space through pairwise comparisons and preference modeling.

The practical implication for systems engineering practice is that Pareto analysis is most powerful when you've done the upstream work to identify which 2–3 objectives are genuinely in tension. This is not a mathematical task — it requires stakeholder interviews, functional decomposition, and the kind of semantic modeling that clarifies which attributes are truly independent versus correlated. A

well-constructed MBSE model, with properly attributed requirements and performance parameters, is exactly the substrate that makes this upstream clarity achievable. Which is why the three methods — RSM for surface approximation, MBSE for structural grounding, Pareto for decision framing — are strongest when used together rather than in isolation.

SysML v1 and v2 are not an incremental update relationship — they represent a fundamental architectural rethink driven by a decade of hard-won lessons about what broke in practice. The OMG finalized SysML v2 as a standard in 2024 after roughly eight years of development, and the changes touch every layer: the metamodel, the syntax, the tooling philosophy, and the relationship to simulation and AI.

SysML v1 (2006–2023)	SysML v2 (2024–present)
FOUNDATION	
UML 2 profile — SysML is a dialect of UML, inheriting its metamodel and all its constraints	KerML (Kernel Modeling Language) — a purpose-built semantic foundation. SysML v2 is defined on top of KerML, not UML clean break
SYNTAX	
Graphical only — diagrams are the primary artifact. No normative textual syntax; every tool invents its own	Normative textual syntax (SysML v2 textual notation) alongside graphical. Models can be written, versioned, and diffed as text git-friendly
TOOL INTEROPERABILITY	
XMI file exchange — brittle in practice. Round-tripping between tools loses fidelity. Vendor lock-in is the norm	REST API standard — tools speak to a shared model repository via HTTP. Any conformant tool can read or write the same model federated
DIAGRAM TYPES	
9 fixed diagram types (BDD, IBD, req, activity, sequence, state machine, use case, parametric, package)	Views and viewpoints — diagram types are not fixed. Tools generate views tailored to stakeholder concerns from the underlying model
REQUIREMENTS	
<<requirement>> stereotype — text in a box with trace relationships. No formal semantics; cannot be machine-verified	RequirementUsage with formal constraint expressions. Requirements can carry satisfaction conditions checkable by analysis tools verifiable
PARAMETRICS / SIMULATION	
Parametric diagram — connects constraint blocks. Useful but limited; not executable without tool-specific plugins	Action semantics and analysis framework built in. Models can be annotated for direct execution or FMI export without plugins executable
TYPING SYSTEM	
Weak typing inherited from UML. Property types often informal; units and quantities not natively enforced	Strong typing with built-in quantity kinds, units (ISO 80000), and value types. Dimensional analysis is model-level rigorous
MATURITY / TOOLING	
Mature ecosystem: Cameo, Rhapsody, Enterprise Architect, Capella all support it. Large installed base, trained practitioners	Standard finalized 2024. Tool support emerging (Syndeia, Cameo v2 beta, open-source SysIDE). Production adoption 2025–2027 early days
AI / KNOWLEDGE GRAPH READINESS	
Models are opaque binary/XMI blobs — not queryable without tool vendor APIs. No semantic web alignment	REST API + JSON-LD potential alignment. Models are first-class queryable resources. Natural fit for knowledge graph integration graph-native

Why v1 accumulated the problems it did

SysML v1 was designed in 2006 as a profile on top of UML 2 — a pragmatic choice that let it leverage UML's existing tooling ecosystem. But that inheritance carried heavy baggage. UML was designed for software, not systems. Its metamodel has concepts (classes, interfaces, components) that map awkwardly onto physical systems with multiple domains, physical properties, and mathematical behavior. Every systems modeler who has tried to represent a thermal-hydraulic system or a power conversion chain in SysML v1 eventually hits the same wall: the language doesn't natively understand units, quantities, or physics.

The deeper problem was the graphical-first philosophy. When diagrams are the primary artifact, the model *is* the picture — and pictures don't compose, version-control cleanly, transform automatically, or feed into downstream analysis tools without manual intervention. Every diagram a systems engineer drew in Cameo was effectively a hand-crafted visualization of an underlying model that lived in a proprietary XML file that only Cameo could faithfully read. Round-tripping to IBM Rhapsody or Enterprise Architect was possible in theory and painful in practice. This created enormous tool lock-in and made multi-vendor program environments — common in large defense and aerospace programs — a constant source of friction.

The KerML foundation — why it matters

KerML (Kernel Modeling Language) is the semantic core that SysML v2 sits on. Its key architectural move is to unify the concepts of *classifier*, *feature*, and *relationship* into a single coherent metamodel without UML's baggage. In KerML everything is either a type (something that classifies instances) or a feature (something that characterizes instances of a type), and relationships are just features with multiple ends. This sounds abstract but it has concrete consequences: it becomes possible to represent the same system element in multiple modeling paradigms (structural, behavioral, parametric) using a single consistent underlying entity rather than copying information across diagram types.

KerML also introduces the concept of *occurrences* — individual snapshots of elements through time — which is the foundation for connecting the structural model to simulation. A part in KerML is not just a static box; it is an entity with a life history that can be sampled, analyzed, and compared against measured data. This is the semantic bridge that makes executable MBSE tractable without tool-specific plugins.

The textual syntax — bigger than it sounds

Giving SysML v2 a normative textual notation is probably the single most consequential practical change for working practitioners. Consider what becomes possible:

You can write a SysML v2 model in a text editor, store it in Git, branch it, merge it with conflict resolution, do pull requests, run automated linting checks, and diff two versions of the system architecture the same way you diff code. None of this is straightforward with SysML v1 XML files. A diff of two Cameo `.mdzip` files is essentially unreadable noise.

You can also write scripts that *generate* SysML v2 model fragments — from requirements databases, from simulation outputs, from knowledge graphs. The model becomes a first-class data artifact that tools can write to, not just read from. This is the same philosophy shift that happened in infrastructure as code (Terraform, Ansible) — moving from GUI-managed state to text-managed state that can be version-controlled and automated.

The REST API — the federated model repository vision

SysML v2's specification includes a normative REST API for model repositories. The intent is that any conformant SysML v2 tool can connect to any conformant repository and exchange model elements through standardized HTTP endpoints — GET a package, POST a new part definition, PATCH a requirement's satisfaction condition.

This is architecturally identical to what you built at Verizon with the knowledge platform, and what the ARCH architecture at AbbVie represents: a federated, API-accessible semantic graph where multiple tools and consumers share a common model rather than each maintaining their own siloed copy. The SysML v2 API spec essentially applies that pattern to engineering model management. The implications for multi-contractor programs are significant — a prime contractor, a subsystem integrator, and a safety analysis vendor can all work against the same model repository with appropriate access controls, rather than exchanging XMI snapshots via email and reconciling differences manually.

Where v2 falls short — honest assessment

The tooling ecosystem is the obvious gap. Cameo Systems Modeler, the dominant SysML v1 tool, has v2 support in beta as of 2024–2025 but production programs are still running v1. The practitioner community — the systems engineers who know SysML v1 fluently — have to learn a significantly different conceptual model and a new syntax. The transition will take years, not months, and there will be a long period of parallel existence where v1 programs are still running while v2 is adopted for new starts.

The REST API specification, while normative, is not yet widely implemented. The ecosystem of conformant repositories is small. The vision of plug-and-play tool interoperability is not yet a lived reality — it's a standard waiting for the tools to catch up.

There is also a genuine complexity increase. KerML's unified metamodel is more expressive and more rigorous than SysML v1's UML profile, but it is also more abstract. A practitioner who learned systems modeling through BDDs and IBDs will find v2's feature/classifier ontology requires a mental model shift. The learning curve is real.

The AI and knowledge graph angle — where this gets interesting for your work

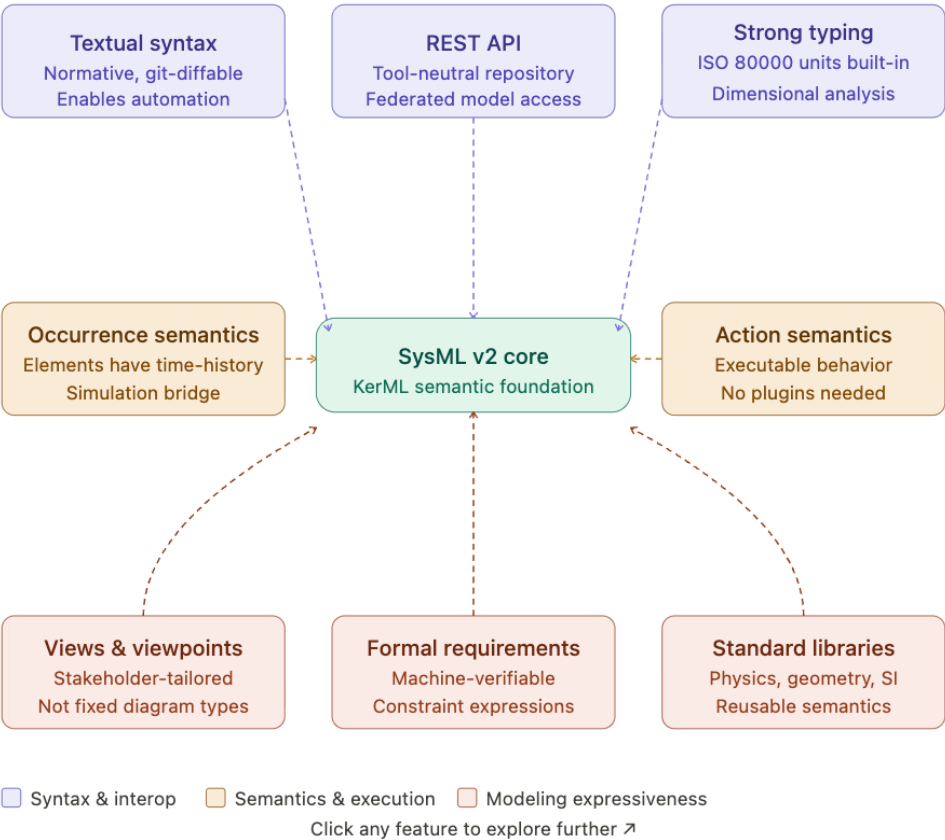
The most underappreciated aspect of SysML v2's architecture is its natural alignment with knowledge graph patterns. A model stored in a REST API repository, with elements identified by URIs and relationships expressed as typed edges, is structurally a labeled property graph — exactly the pattern of your ARK/Neo4j work at AbbVie. The potential to connect an SysML v2 model

repository to an enterprise knowledge graph, so that a part definition in the engineering model is the same node as the corresponding record in the PLM system and the safety analysis database, is architecturally achievable in a way it simply was not with SysML v1's XMI blobs.

This also opens a genuinely interesting door for AI grounding in engineering contexts. An LLM or AI assistant querying a SysML v2 repository via its REST API could retrieve the actual model — verified, requirements-traced, simulation-validated — rather than hallucinating structural relationships from training data. The Circuit Breaker pattern you've developed applies directly here: the AI's architectural claims are grounded against the authoritative model graph, and the MTBH metric becomes measurable because you have a formal representation of what is true to compare against what the AI asserted.

SysML v2, in other words, is not just a modeling language upgrade. It is an attempt to make engineering knowledge a first-class, queryable, federated resource — which is exactly the direction that serious knowledge engineering practice has been heading independently.

SysML v2's most important features aren't independent additions — they form a coherent architectural philosophy. Let me walk through the features that matter most in practice, grouped by what problem each solves.



1. The textual notation — models as text

This is the feature with the highest practical leverage. SysML v2 ships a normative textual syntax alongside the graphical notation. A simple part definition looks like this:

```
part def Vehicle {  
    part engine : Engine;  
    part transmission : Transmission;  
    part wheels[4] : Wheel;  
  
    attribute mass : MassValue;  
    attribute topSpeed : SpeedValue;  
}
```

This is not pseudo-code — it is the actual SysML v2 textual notation, from which a conformant tool can construct the full model. The consequences cascade: models can live in Git repositories, be diffed line-by-line, be generated by scripts, be validated by CI pipelines, and be written in any text editor. The [syside](#) VSCode extension (open source) already provides syntax highlighting and basic validation. This is the infrastructure-as-code moment for systems engineering — the same philosophical shift that moved cloud infrastructure from GUI-click-driven to Terraform-managed.

2. Strong typing with built-in quantities and units

SysML v1 had weak typing inherited from UML. You could declare a property called `mass` of type `Real` and another called `velocity` of type `Real` and the model had no way to know these were dimensionally incompatible. SysML v2 ships standard libraries that define quantity kinds and value types conformant to ISO 80000 (the international standard for quantities and units):

```
attribute mass : MassValue = 1500 [kg];  
attribute velocity : SpeedValue = 27.8 [m/s];  
attribute kineticEnergy : EnergyValue = 0.5 * mass * velocity**2;
```

A conformant tool can now perform dimensional analysis — catching a modeler who accidentally adds a mass to a force, or assigns a temperature where a pressure is expected. For complex multi-domain systems (thermal + structural + electrical + controls), this prevents a whole class of interface errors that currently propagate silently through SysML v1 models into simulation setups.

3. Occurrence semantics — elements with time-history

This is the most philosophically significant change and the least understood. In SysML v1, a part or component is essentially a static box — it has properties and connections, but no inherent notion of existing through time. In SysML v2's KerML foundation, every element is an *occurrence* — something that exists over a span of time and has a life history. A `PartUsage` is not just a structural placeholder; it is an entity that starts, persists, changes state, and ends.

This matters because it is the semantic bridge to simulation. A simulation is precisely a computation over the time-history of system occurrences — tracking how their attributes evolve as actions execute and events occur. When the model already encodes the temporal nature of its elements, connecting it to a simulation engine becomes a matter of mapping, not translation. SysML v1's parametric diagram was a weak attempt at this; occurrence semantics makes it a first-class architectural commitment.

4. Action semantics — executable behavior without plugins

SysML v1's activity diagrams described behavior graphically but were not executable — a tool had to interpret them with proprietary plugins to do anything computational. SysML v2 defines action semantics directly in the specification: control flow, data flow, conditional branching, iteration, and parallel execution all have precise computational meanings. A compliant model can be executed by a compliant engine without vendor-specific interpretation.

The combination of occurrence semantics and action semantics means a SysML v2 model can, in principle, be directly simulated — or exported to FMI/FMU format — without the manual handoff to Simulink or Modelica that v1 workflows require. In practice this capability is still maturing in the tooling, but the specification lays the foundation that v1 never had.

5. The REST API and model repository standard

SysML v2 specifies not just the language but the *infrastructure* for storing and exchanging models. The API defines standard endpoints:

- `GET /projects/{projectId}/commits/{commitId}/elements` — retrieve all model elements at a specific commit
- `POST /projects/{projectId}/commits` — create a new commit with model changes
- `GET /projects/{projectId}/commits/{commitId}/elements/{elementId}/relationships` — retrieve all relationships of an element

The commit-based model is deliberate — it mirrors Git semantics. Every change to the model creates a new commit, preserving the full history. This means you can audit what the system architecture looked like at any point in the program, compare branches (what would change if we adopted Design Option B?), and merge contributions from distributed teams.

For your ARK/Neo4j work and the federated knowledge graph architectures you've built: a SysML v2 repository is structurally a labeled property graph over HTTP. The elements are nodes, relationships are edges, and the API is a graph query interface. Connecting a SysML v2 repository to a Neo4j instance — or to an enterprise knowledge graph — is a straightforward ETL problem, not a semantic translation challenge. This is architecturally what the ARCH pattern aspires to, applied to engineering models.

6. Views and viewpoints — replacing fixed diagram types

SysML v1's nine diagram types were a taxonomy imposed on the modeler. You had BDDs for block structure, IBDs for internal connections, activity diagrams for behavior — and if your stakeholder needed to see something that didn't fit a standard diagram, you were improvising.

SysML v2 replaces fixed diagram types with a *viewpoint/view* mechanism. A viewpoint is a specification of what a particular stakeholder role cares about — what elements are relevant, what relationships are shown, what attributes are exposed. A view is a rendering of the model filtered through a viewpoint. Tool vendors implement viewpoint definitions, and organizations can define their own. The model is the authority; views are projections of it for specific audiences. This is the same principle as a database view — the underlying data is unchanged, the presentation is tailored.

7. Formal requirements with machine-verifiable satisfaction

SysML v1 requirements were text in a box with `<<satisfy>>` and `<<verify>>` relationships drawn as lines. These were documentary — a human read them and judged satisfaction. SysML v2 `RequirementUsage` can carry formal constraint expressions:

```
requirement vehicleRangeReq {  
  subject v : Vehicle;  
  require constraint { v.range >= 500 [km] }  
}
```

A conformant analysis tool can now evaluate whether a given design satisfies this requirement by checking the constraint against the model's attribute values — or against simulation outputs. Requirements become executable specifications, not just documentation. For programs operating under DO-178C, ARP4754A, or ISO 26262, this is the foundation for model-based verification evidence rather than document-based verification evidence.

8. Standard libraries

SysML v2 ships with a set of standard libraries that are part of the specification — not tool add-ons. These include the SI Units library (masses, lengths, times, temperatures, currents, and their derived quantities), a Geometry library, a Sequence Functions library, and domain libraries for common engineering patterns. This means two organizations using different conformant tools will share the same definitions of `MassValue` and `ForceValue` — eliminating the silent semantic drift that happens when every program defines its own unit types.

The compound effect

The reason these features matter more together than separately is that they create a closed loop the field has never had before. A requirement written as a formal constraint (feature 7) references typed attributes (feature 2) on parts with temporal behavior (feature 3) driven by executable actions (feature 4). The resulting model lives in a versioned repository (feature 5) accessible via API to simulation tools, analysis tools, AI systems, and downstream consumers. Stakeholder-specific views (feature 6) are generated from the same underlying model that traces back to source

requirements. Everything is written in text (feature 1) that can be generated, validated, and evolved programmatically.

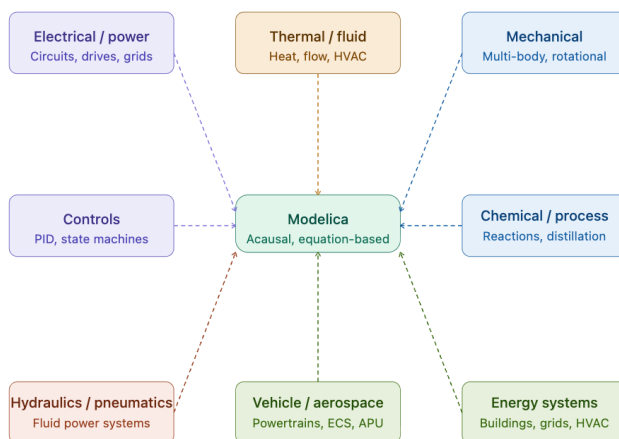
This is the vision of a *living system model* — one that remains the authoritative artifact throughout the program lifecycle rather than becoming a static snapshot that diverges from the real system the moment detailed design begins. Whether the tooling ecosystem catches up to the specification quickly enough to make this vision practical at scale is the open question. The specification, for the first time, makes it architecturally achievable.

Modelica occupies a distinctive niche in the simulation landscape: it is the language of choice when your system spans multiple physical domains simultaneously and you care about the continuous-time dynamics of the whole — not just one subsystem in isolation. Understanding where it excels and where it breaks requires understanding what makes it architecturally different from every other simulation approach.

The core architectural idea: equation-based, acausal modeling

Every other mainstream simulation environment — Simulink, LabVIEW, bond graphs in most tools — is *causal*: you wire blocks together in signal flow direction, explicitly specifying what is input and what is output. Modelica is *acausal*: you write equations describing physical relationships, and the compiler figures out the causality automatically through symbolic manipulation.

This distinction is deeper than it sounds. Consider a simple electrical resistor. In a causal tool you write either $V = I * R$ (voltage as output given current as input) or $I = V / R$ (current as output given voltage as input) — you commit to one direction before you know how the resistor will be used in a circuit. In Modelica you write $v = i * R$ and connect the component's terminals. The compiler analyzes the full system of equations, determines which variables are unknown at each point, and selects the appropriate computational sequence automatically. The same resistor model works whether it is the driven element or the driving element in the circuit.



All domains share a single equation system — interactions are automatic ↗
Click any domain to explore further

Where Modelica is genuinely the best tool

Multi-domain physical systems are Modelica's home territory. An electric vehicle powertrain involves electrical energy storage (battery electrochemistry), power electronics (switching circuits), electric motors (electromechanical coupling), mechanical drivetrain (rotational dynamics, gear ratios, wheel-road contact), thermal management (coolant flow, heat exchangers), and controls (motor torque commands, battery management). In Simulink you would build each subsystem separately and wire them together through signal interfaces — which requires you to decide, up front, which quantities flow as signals and which direction information travels. In Modelica you write the physics of each component and connect terminals. The compiler assembles the full differential-algebraic equation (DAE) system and solves it as a whole, which means cross-domain interactions — the heat generated by the motor affecting its own resistance, for example — emerge naturally rather than requiring explicit feedback loops.

Automotive is where Modelica has its deepest industrial adoption. BMW, Toyota, Daimler, and virtually every major OEM use Dymola or OpenModelica for vehicle system simulation. The Modelica Standard Library ships a complete `Modelica.Mechanics` package covering translational and rotational mechanics, a `Modelica.Electrical` package for analog and digital circuits, `Modelica.Thermal` for heat transfer, and `Modelica.Fluid` for one-dimensional fluid flow. Third-party libraries extend this to vehicle dynamics (Modelon's Vehicle Dynamics Library), powertrains (LMS Imagine.Lab), and batteries (Battery library).

Building energy simulation is a fast-growing domain. Lawrence Berkeley National Laboratory's Buildings library is a major open-source Modelica library used by the US Department of Energy for HVAC optimization, district energy modeling, and building controls research. The acausal approach is ideal here because a building's thermal, electrical, and HVAC systems are tightly coupled — a change in occupancy patterns affects heating loads which affects electrical demand which affects controls — and these interactions are exactly what you want the solver to handle automatically.

Process engineering and chemical plants are increasingly using Modelica through libraries like Modelon's Thermal Power Library and the ThermoSysPro library developed by EDF for nuclear and thermal power plant simulation. The ability to model phase changes, reaction kinetics, and heat integration in a single equation system makes it competitive with domain-specific tools like Aspen Plus for dynamic simulation (though Aspen remains stronger for steady-state process design and its built-in thermodynamic property databases are more mature).

Aerospace environmental control systems, hydraulics, and fuel systems are well-served. Airbus and DLR (German Aerospace Center) have been major Modelica contributors. DLR's flight dynamics library covers aerodynamics, propulsion, and control surface modeling. The European Space Agency uses Modelica for launcher system simulation.

What a Modelica model actually looks like

A Modelica component definition for a simple thermal mass:

```
modelica
model ThermalMass
  parameter Real C = 1000 "Heat capacity [J/K]";
  Real T(start=293.15) "Temperature [K]";
  Modelica.Thermal.HeatTransfer.Interfaces.HeatPort_a port;
equation
  C * der(T) = port.Q_flow;
  port.T = T;
end ThermalMass;
```

The equation `C * der(T) = port.Q_flow` says: heat capacity times rate of temperature change equals heat flowing in at the port. The component doesn't know whether it will be connected to a heater, a heat exchanger, or ambient air — the compiler resolves that when you assemble the system. `der(T)` is Modelica's notation for the time derivative dT/dt , which the solver integrates numerically.

The Modelica Standard Library and ecosystem

The Modelica Standard Library (MSL) ships with every Modelica tool and provides approximately 1,400 model components across mechanics, electrical, thermal, fluid, control, and math domains. It is the shared vocabulary that makes Modelica models portable across tools. Beyond the MSL, a rich ecosystem of commercial and open-source libraries covers specific domains — the Modelon Impact platform hosts a library marketplace, and the Modelica Association maintains a library portal.

Where Modelica struggles — the honest limitations

High-frequency switching and power electronics push Modelica solvers into difficulty. A switching converter that opens and closes at 100kHz creates discontinuities in the equation system at every switching event. Modelica handles events through a reinitialize-and-restart mechanism, but at high switching frequencies the solver spends most of its time restarting rather than integrating — simulation speed degrades badly. SPICE-based tools like LTspice or Cadence are far better for detailed power electronics design. Modelica is better treated as the system-level framework that receives averaged or behavioral models of power electronics rather than detailed switching models.

Spatially distributed phenomena — anything requiring partial differential equations (PDEs) — are outside Modelica's native scope. Modelica is fundamentally a tool for lumped-parameter or one-dimensional distributed systems (pipes, beams). A full 3D thermal or structural analysis requires FEA tools like ANSYS or Abaqus. The standard approach is to use FEA for the detailed spatial analysis and export a reduced-order model (ROM) into Modelica for system-level integration. This is sound engineering but requires the ROM generation step as an additional workflow.

Discrete-event and agent-based systems are a poor fit. If you are simulating a manufacturing line, a logistics network, a communications protocol, or any system where the interesting dynamics are in the sequencing of discrete events rather than continuous physical evolution, Modelica is the wrong tool. AnyLogic, SimPy, or MATLAB's SimEvents are better choices. Modelica has some

discrete-event capability through its event handling, but it was not designed for this and shows.

Initialization complexity is a notorious practical challenge. Because Modelica's compiler automatically determines the computational sequence from the equation system, initializing that system — specifying enough starting conditions that the DAE has a unique solution — can be non-obvious. Large models with many components can fail to initialize with cryptic error messages about singular systems or underdetermined conditions. Experienced Modelica users develop intuition for what initialization conditions are needed, but this is a genuine barrier for new users and a source of significant debugging time on complex models.

Solver selection and stiffness require expertise. The DAE systems Modelica generates are often stiff — they contain dynamics operating on very different time scales simultaneously (a fast electrical transient and a slow thermal drift in the same model). Stiff DAE solvers like DASSL, IDA, or Radau are necessary, but selecting appropriate solver tolerances, maximum step sizes, and handling of algebraic loops requires understanding numerical methods. A poorly configured solver produces wrong answers quietly, which is the most dangerous failure mode in simulation.

Proprietary tool fragmentation remains a problem despite the open standard. Dymola is the commercial benchmark — its compiler is widely considered the most robust — but it is expensive and its licensing model reflects that. OpenModelica is free and increasingly capable but has gaps in solver robustness and library compatibility. JModelica.org (now Optimica Compiler Toolkit, commercial) and Modelon Impact are serious alternatives. Models written for one tool sometimes require modification to run on another due to differences in library versions, solver interfaces, and language extensions. The FMI/FMU export standard partially addresses this by letting you export a compiled model that runs anywhere, but the source-level portability is imperfect.

The connection back to MBSE and RSM

Modelica sits precisely at the junction of the three methods from our earlier discussion. In a mature MBSE workflow, SysML v2 holds the structural architecture and requirements; Modelica implements the physics of each subsystem as equation-based components; RSM builds surrogate models by sampling Modelica's outputs at DoE-selected operating points; and the surrogate feeds a Pareto optimization over competing system objectives. Modelica is the physics engine — the source of ground truth that makes the surrogates meaningful and the optimization results trustworthy. Its limitations are the places where that ground truth is either too expensive to compute (high-frequency switching), structurally inaccessible (PDEs), or simply outside the physical modeling paradigm (discrete events) — and recognizing those boundaries is what determines whether you're using the tool well or fighting it

AADL (Architecture Analysis and Design Language) is a standards-based modeling language developed originally by the Society of Automotive Engineers (SAE) and standardized as AS5506. It was designed from the ground up for a specific problem that SysML handles poorly: formally specifying and analyzing the *runtime architecture* of embedded, safety-critical software-hardware

systems — and doing so with enough mathematical precision that timing, scheduling, safety, and reliability properties can be *proved*, not just estimated.

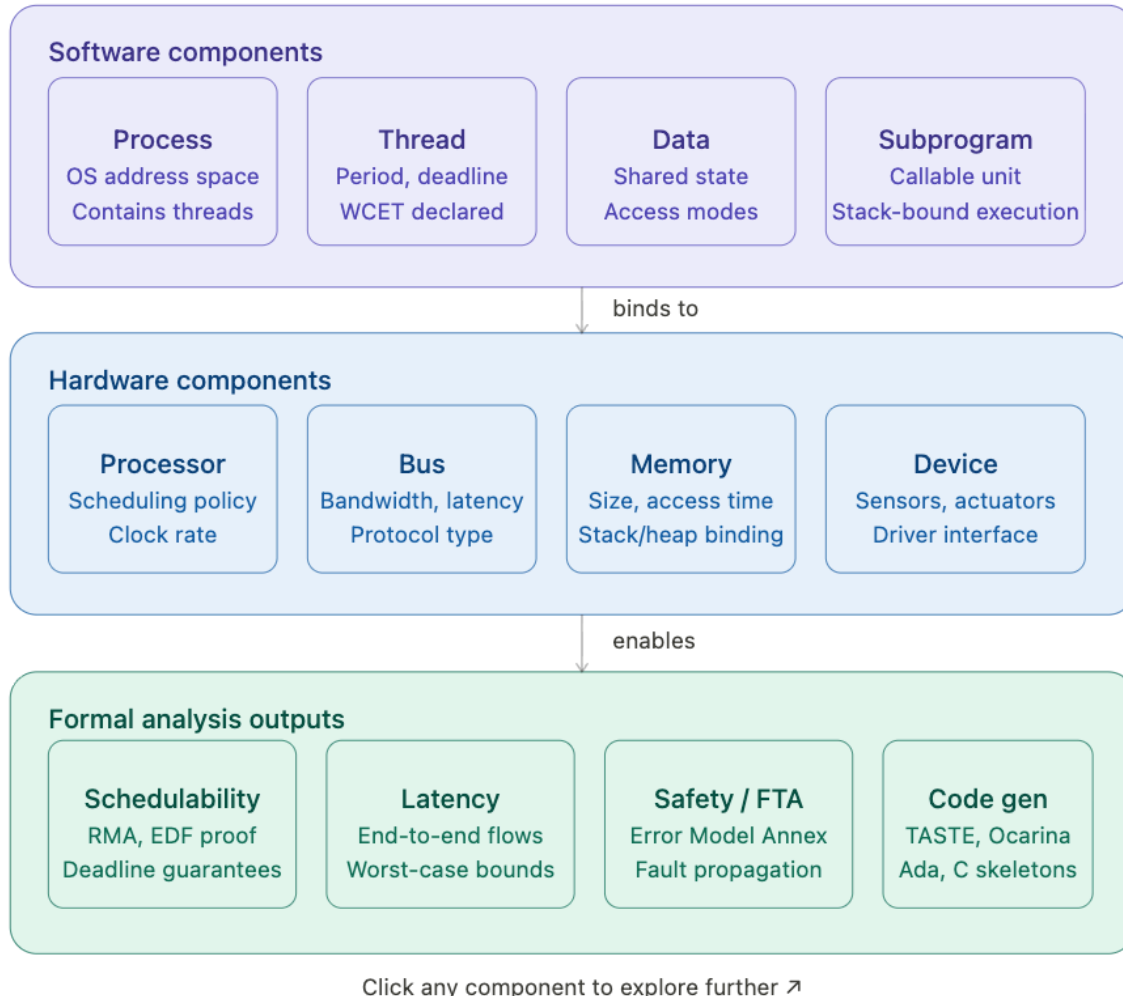
The clearest way to understand AADL is to understand what question it answers. SysML answers "what is the system and how is it structured?" AADL answers "will this system's software execute correctly on this hardware, within these timing constraints, without violating these safety properties?" These are different questions, and the language you use to answer them has to be designed accordingly.

The fundamental architectural commitment

AADL models a system as a composition of *components* — software components (processes, threads, subprograms, data), hardware components (processors, memory, buses, devices), and composite components that bridge both (systems). Every component has explicitly declared *properties*: execution time ranges, periods, deadlines, memory footprints, data access modes, communication protocols, fault modes.

The critical move is that these properties are not annotations for human readers — they are inputs to formal analysis tools. When you declare that a thread has a period of 10ms and a worst-case execution time of 3ms on a specific processor, AADL's analysis ecosystem can determine whether the full task set is schedulable, whether any thread will miss its deadline, and whether data shared between threads is accessed safely. This is *schedulability analysis* — a mathematical proof that the system will meet its timing requirements — and it is AADL's killer capability.

SysML has no equivalent. A SysML parametric diagram can express that execution time must be less than some value, but it cannot prove that a specific software architecture running on specific hardware will actually meet that constraint at runtime



What AADL looks like in practice

AADL has both a textual and a graphical notation. The textual form is the normative one, and it reads like a typed architecture description language. A fragment modeling a flight control thread bound to a processor:

```
aadl
thread FlightControlThread
  features
    sensorInput : in data port SensorData.impl;
    actuatorCmd : out data port ActuatorCommand.impl;
  properties
    Dispatch_Protocol => Periodic;
    Period            => 20ms;
```

```
Deadline      => 20ms;
Compute_Execution_Time => 2ms .. 4ms;
Priority       => 10;
end FlightControlThread;

processor FlightComputer
  properties
    Scheduling_Protocol => (Rate_Monotonic);
    Clock_Period       => 1ms;
  end FlightComputer;

system FlightControlSystem
  ...
  properties
    Actual_Processor_Binding => (reference FlightComputer)
    applies to flightControlThread;
  end FlightControlSystem;
```

Every property here is an input to analysis. Declare that this thread has a 20ms period, a 4ms worst-case execution time, and a Rate Monotonic scheduler, and AADL tooling can compute the utilization bound and determine whether the full task set is schedulable. This is not an estimate — it is a mathematical certificate under Rate Monotonic Analysis assumptions.

The analysis ecosystem

AADL's value proposition is entirely in its analysis tools — the language without them would be just another architecture description notation. The key tools:

OSATE (Open Source AADL Tool Environment) is the reference implementation, built on Eclipse, maintained by Carnegie Mellon's Software Engineering Institute. It provides schedulability analysis, end-to-end flow latency analysis, and a plugin architecture for additional analyses. The Error Model Annex (EMV2) plugin enables fault tree analysis and failure mode propagation directly from the architectural model — so you can trace how a sensor failure propagates through the system and affects safety-critical outputs, all without leaving the architecture model.

TASTE (the Tools for Asynchronous Systems and Time-sensitive Environments) is ESA's open-source toolchain built on AADL. It takes an AADL model and generates deployment infrastructure — task frameworks, communication middleware, scheduling configuration — in Ada or C, from which engineers write only the application logic. This is AADL's code generation story: the architecture is specified formally, and the scaffolding is generated rather than hand-coded.

Ocarina is a command-line AADL processor that supports code generation and model transformation. It is used in research contexts and in programs where the Eclipse-based OSATE is too heavyweight.

When to use AADL instead of SysML

The decision comes down to what analysis you need to perform and at what level of architectural specificity you are working.

Use AADL when you need to *prove* timing properties. If your system has hard real-time constraints — avionics, engine control units, autonomous vehicle safety systems, medical device controllers — and you need to demonstrate to a certification authority (DO-178C, IEC 61508, ISO 26262) that those constraints are met by the architecture, AADL gives you the formal machinery to do it. SysML gives you documentation. The difference matters enormously in a certification context.

Use AADL when the SW/HW binding is the architectural problem. AADL explicitly models how software components are bound to hardware resources — which threads run on which processors, which data flows over which buses, what memory is allocated where. When you are making partitioning decisions for an IMA (Integrated Modular Avionics) system, or allocating tasks across a multicore processor under ARINC 653, AADL's binding model is exactly the right abstraction. SysML has no equivalent.

Use AADL when you are working in a DO-178C or ARP4754A context and your certification evidence strategy is model-based. OSATE-generated analysis reports and flow latency certificates have been accepted as evidence by the FAA and EASA in certification programs. This is a mature, proven path.

Use SysML instead when you are working at mission/system level above the software architecture. SysML is the right language for capturing stakeholder requirements, defining system context, allocating functions to subsystems, and managing the system decomposition that feeds downstream engineering. SysML's breadth across multiple viewpoints (requirements, structure, behavior, parametrics) makes it the natural hub for early-phase systems engineering. AADL enters later, when the software architecture is being detailed.

Use SysML instead when your system is not primarily a real-time embedded software system. A satellite's thermal control system, a pharmaceutical manufacturing line, or a power grid protection scheme all have real-time elements — but the dominant modeling concerns are physical, not computational. SysML feeds Modelica or FEA tools for those. AADL would be a poor fit.

The combination pattern in mature programs

The most sophisticated avionics and defense programs use both, at different levels and for different concerns. SysML models the system architecture — the aircraft's functions, the avionics system's requirements, the allocation of functions to LRUs (Line Replaceable Units). AADL models the software architecture within each LRU — the threads, processes, scheduling, and communications that implement the allocated functions. A traceability relationship connects the SysML function allocation to the AADL component that implements it, closing the verification loop.

This mirrors the broader pattern from our earlier discussion: SysML v2's federated model repository and AADL's formal analysis outputs are complementary assets that should share a common

traceability backbone — ideally a knowledge graph that makes cross-model queries possible. A question like "show me all safety-critical threads whose worst-case execution time exceeds 80% of their period, and trace them back to the system-level safety requirements they implement" is answerable if and only if the SysML and AADL models are nodes in the same graph with traversable relationships between them. That integration is the frontier that neither tool has solved alone — and it is precisely the architectural problem your ARCH work and the Circuit Breaker framework are positioned to address.

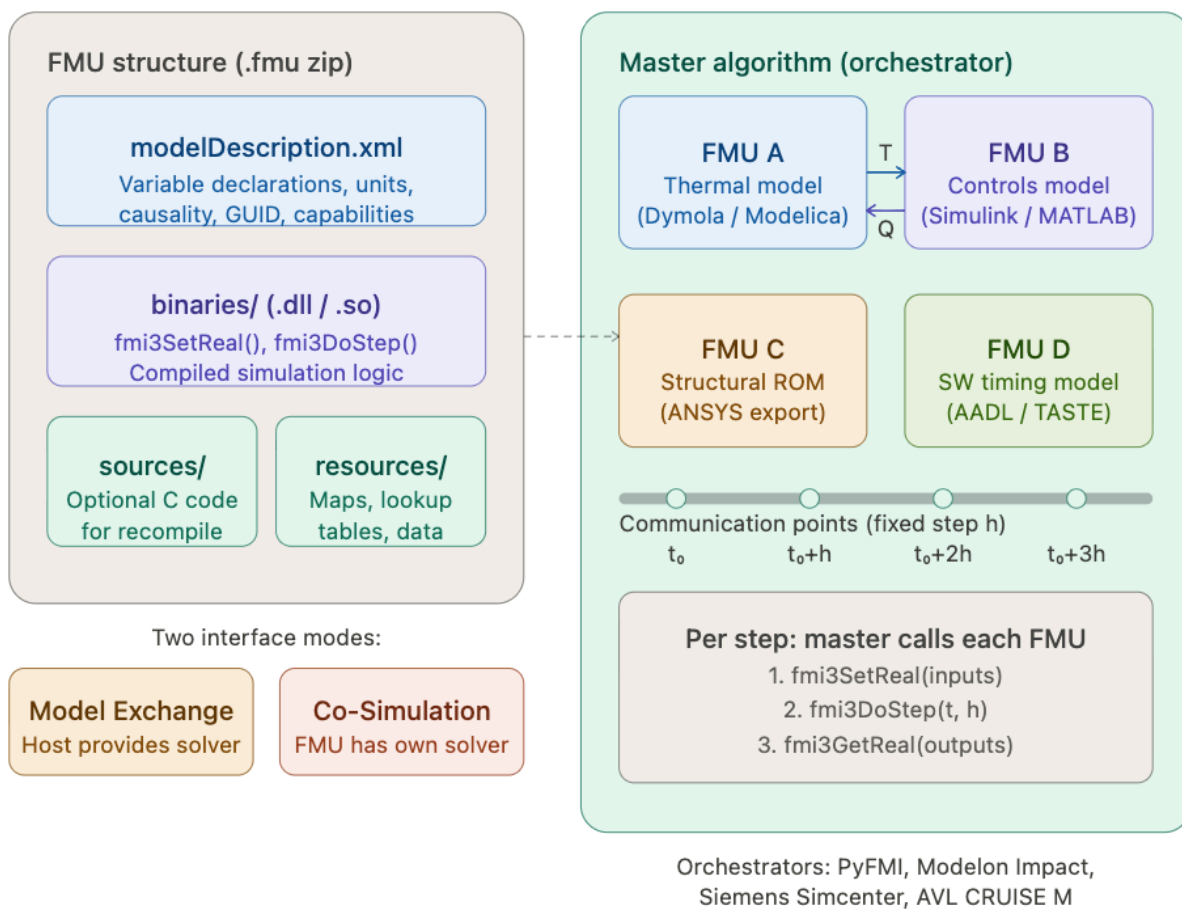
FMI is the engineering equivalent of a USB standard — a vendor-neutral contract that lets simulation components from completely different tools plug into each other without either side needing to know anything about the other's internals. Before FMI existed, connecting a Modelica thermal model to a Simulink controls model to an ANSYS structural model required custom glue code, format translators, and fragile point-to-point integrations that broke every time a tool version changed. FMI replaces all of that with a single, stable interface specification.

The standard is maintained by the Modelica Association (despite the name, FMI is tool-agnostic and not specific to Modelica) and has gone through three major versions: FMI 1.0 (2010), FMI 2.0 (2014, the widely deployed version), and FMI 3.0 (2022, adding clocked variables, array support, and binary data).

The core abstraction: the Functional Mockup Unit

An FMU is a self-contained simulation component — a zip archive with a specific internal structure. It contains the compiled simulation code (a shared library: `.dll` on Windows, `.so` on Linux), an XML model description file that declares all inputs, outputs, parameters, and state variables, and optionally source code. The compiled library exposes a C API — a set of standardized function calls — that any host environment can invoke.

The model description XML is the semantic contract. It declares every variable the FMU exposes: its name, data type, causality (input/output/parameter/state), variability (constant/tunable/discrete/continuous), and physical unit. A host environment reads this XML at import time and knows exactly what the FMU expects and provides, without executing any simulation code.



Click any element to explore further ↗

The two interface modes

FMI defines two fundamentally different ways a component can participate in simulation, and understanding the distinction matters for how you architect a co-simulation system.

Model Exchange exports the model's equations without a solver. The FMU exposes its state derivatives and algebraic equations, and the host environment — the importing tool — integrates those equations using its own numerical solver. This produces tighter numerical coupling: the host solver can adapt its step size to handle stiff dynamics, can compute Jacobians across multiple FMUs simultaneously, and can maintain higher-order accuracy. The cost is that the host must have a solver capable of handling the FMU's equation structure, and all FMUs in a Model Exchange configuration must share the same solver.

Co-Simulation packages the solver inside the FMU. Each FMU advances its own internal state from

time t to $t+h$ using whatever solver its original tool used — Runge-Kutta, DASSL, an implicit integrator for stiff systems. The host (the master algorithm) coordinates the exchange of variables between FMUs at communication points spaced h apart, but does not touch the numerical integration. This produces looser coupling — information is only exchanged at communication points, introducing an approximation error proportional to h — but it means each FMU retains its native solver, which is often better tuned for its specific physics than a generic host solver would be.

The vast majority of industrial FMI usage is Co-Simulation mode, because it preserves the solver expertise embedded in each originating tool and requires less of the host environment. Model Exchange is used when numerical accuracy at interfaces is critical and all tools support it.

The master algorithm: what the orchestrator actually does

The master algorithm is the piece most engineers underestimate. At each communication point it must: determine the correct ordering of FMU updates (which FMU's outputs become which other FMU's inputs), handle algebraic loops (circular dependencies where FMU A's output depends on FMU B's output which depends on FMU A's output), manage step size, and detect and handle events (discontinuities in state variables, like a valve opening or a contact making).

For systems without algebraic loops and without strong numerical coupling, the simplest master is a Jacobi scheme: at each step, read all FMU outputs from the previous step, set all FMU inputs, advance all FMUs in parallel, collect outputs. This is explicit and parallelizable but introduces a one-step delay at every interface — which can cause instability in tightly coupled systems where the coupling strength exceeds a threshold related to the communication step size.

For tightly coupled systems, a Gauss-Seidel scheme updates FMUs sequentially, passing outputs from earlier FMUs as inputs to later ones within the same time step. This reduces the delay error but requires knowing the dependency ordering, introduces sequential execution, and can still be unstable for bidirectional coupling (thermal $\leftrightarrow$ structural, for instance, where heat causes deformation which changes heat transfer coefficients).

The most rigorous masters use extrapolation and iteration: at each communication point, they extrapolate FMU inputs based on recent output history, advance all FMUs, compare outputs to extrapolated values, and iterate until convergence. This is Newton-style co-simulation — significantly more expensive but necessary for strongly coupled systems. PyFMI's master implementations and Modelon Impact's orchestration engine support this. The tradeoff is computation cost versus coupling accuracy, and the right choice depends on the physics of the interfaces.

What gets wrapped as FMUs in real programs

The breadth of what can be exported as an FMU in production contexts is wider than most people expect.

Modelica models from Dymola or OpenModelica export as FMUs natively — this is the canonical use case the standard was designed around. A complete vehicle thermal management system

modeled in Modelica becomes an FMU that a Simulink controls engineer can drop into their model as a black box.

Simulink models export as FMUs through MathWorks' FMI Kit or directly from Simulink's export menu. A controls model developed in Simulink — PID tuning, state estimation, supervisor logic — becomes an FMU that a Modelica or Dymola environment can drive.

ANSYS and Nastran can export reduced-order models (ROMs) as FMUs. A full finite element structural model is computationally too expensive to run in real-time co-simulation, so the workflow is: run the FEA offline to generate a ROM (a low-dimensional approximation of the structural dynamics), export that ROM as an FMU, and integrate it into the co-simulation. The ROM loses spatial detail but retains the dominant dynamic behavior at a fraction of the computational cost.

AADL/TASTE models can generate FMUs that represent the timing behavior of the software architecture — modeling how long computations take, when data is available, and how scheduling affects end-to-end latency — without executing the actual flight software. This lets systems engineers test whether software timing assumptions are compatible with plant dynamics before the software exists.

Physical test rigs can be wrapped as FMUs through hardware-in-the-loop (HiL) interfaces. A real hydraulic actuator in a test lab, connected through a dSPACE or National Instruments real-time system, can participate in a co-simulation alongside purely virtual FMUs — mixing physical and virtual components in the same simulation framework.

The communication step size problem — where co-simulation breaks

The step size h between communication points is the central numerical challenge of FMI co-simulation. Too large and the exchanged values are stale — the outputs that FMU B uses as inputs to compute its next step are from h seconds ago, and for fast dynamics that delay introduces significant error or instability. Too small and the overhead of the communication protocol (function calls across the C API, data marshaling) dominates simulation time.

The rule of thumb is that h should be at least ten times smaller than the fastest dynamic time constant at any interface. If the thermal-mechanical coupling has a time constant of 1 second, $h = 0.1$ seconds is usually adequate. If you are coupling a fast electrical transient (time constant 1ms) to a slower mechanical system, you either need $h = 0.1\text{ms}$ (expensive) or you decouple the domains and handle the electrical dynamics inside a single FMU rather than across an FMI interface.

FMI 3.0 introduced *clocked variables* specifically to address this — allowing FMUs to declare variables that are only valid at specific time instants rather than being continuous, which models sampled-data systems (digital controllers, CAN bus messages) more accurately and allows the master to synchronize communication points to physical clock events rather than using a fixed step.

FMI and the broader MBSE/RSM/Pareto picture

FMI is the integration fabric that makes the combination of MBSE, Modelica, ANSYS, and AADL

into a coherent simulation system rather than a collection of siloed tools. In the integration pattern we discussed earlier: the SysML v2 model defines system architecture and interfaces; Modelica, Simulink, and FEA tools implement subsystem physics; FMI wraps each subsystem as an FMU; the co-simulation master assembles them into a virtual integrated vehicle; RSM samples that co-simulation at DoE-selected operating points to build a surrogate model; and Pareto optimization sweeps the surrogate to map the trade-off frontier.

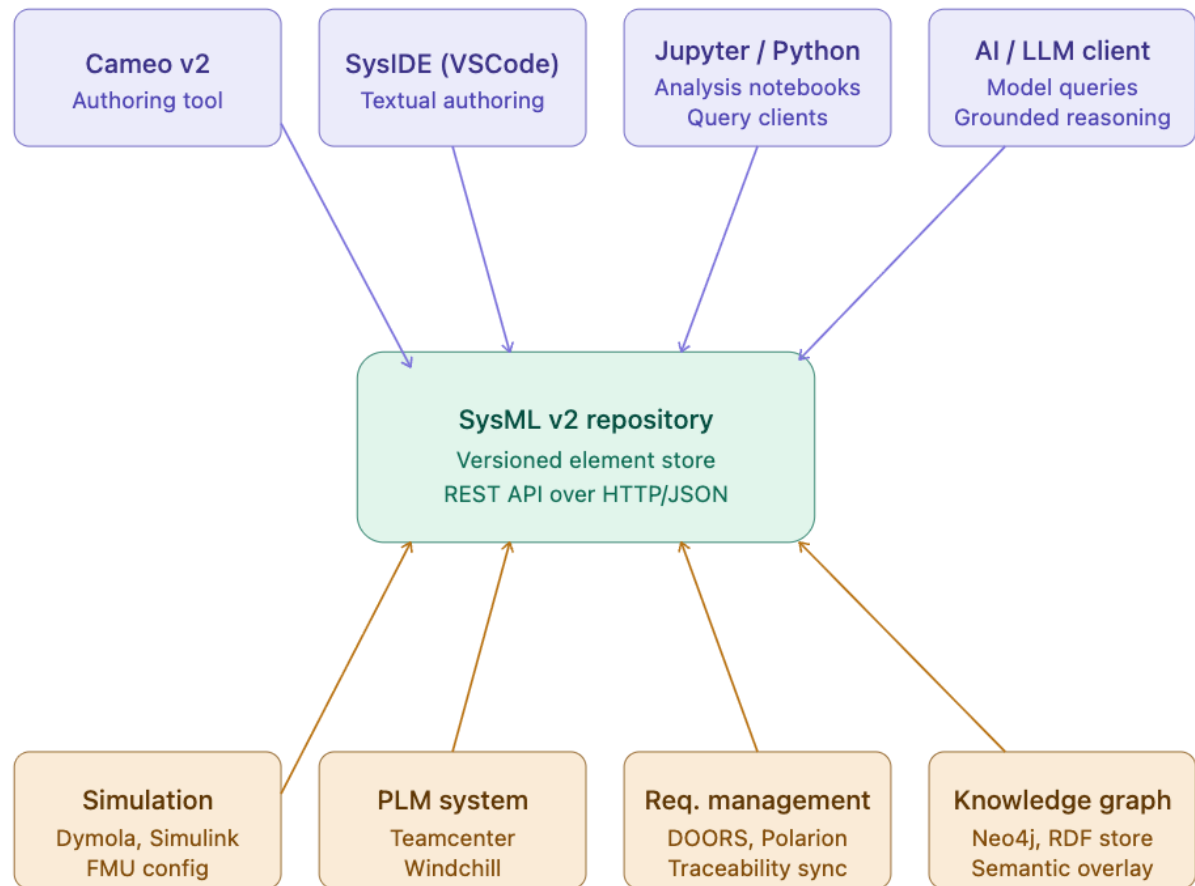
FMI is also what makes that entire chain tractable from a knowledge management perspective. The `modelDescription.xml` of each FMU is a machine-readable semantic declaration of that component's interface — its variables, units, causality, and dependencies. Ingest those XML files into a knowledge graph alongside the SysML v2 model elements and AADL component specifications, and you have a federated, queryable map of the entire simulation system: which FMU provides which physical quantity, which FMU consumes it, what units are expected, what the valid operating range is. This is the grounding layer that makes AI-assisted simulation setup — and AI-assisted detection of interface mismatches — technically feasible rather than aspirational.

The SysML v2 API is arguably the most consequential part of the v2 specification — more so than the new syntax or the KerML metamodel — because it is the piece that transforms MBSE from a single-vendor, file-based discipline into a potentially federated, tool-neutral ecosystem. Every other improvement in v2 is valuable within a tool; the API is what makes value flow *between* tools.

What the API actually is

The SysML v2 API specification defines a RESTful HTTP interface for a *model repository* — a server that stores SysML v2 model elements and exposes them through standardized endpoints. It is not a file format. It is not an export mechanism. It is a live, queryable, writable service that any conformant client can interact with, regardless of what tool created the model content.

The specification covers four conceptual areas: project and version management (how models are organized and how changes are tracked), element CRUD operations (creating, reading, updating, deleting model elements), relationship traversal (navigating connections between elements), and query execution (filtering and searching the model content). Underneath all of this is a commit-based version model that mirrors Git semantics — every change creates an immutable commit, the full history is preserved, and branches are first-class concepts



All connections are standard HTTP/JSON — no vendor adapters ↗

The API endpoints — what the calls actually look like

The specification defines endpoints organized around five resource types. Here is what real API interactions look like, using the reference server implementation that the SysML v2 team maintains as open source.

Project and commit management:

```
http
GET /projects
POST /projects
GET /projects/{projectId}/commits
POST /projects/{projectId}/commits
GET /projects/{projectId}/commits/{commitId}
```

A commit in SysML v2 terms is a snapshot of a set of model changes — additions, modifications, or deletions of elements. Just as a Git commit captures a diff against the previous state, an API commit captures which elements changed and how. The commit carries a timestamp, an author identity, and a reference to its parent commit, forming an immutable chain of model history.

Element operations within a commit:

http

GET /projects/{projectId}/commits/{commitId}/elements

GET /projects/{projectId}/commits/{commitId}/elements/{elementId}

POST /projects/{projectId}/commits (body contains element changes)

GET /projects/{projectId}/commits/{commitId}/elements/{elementId}/relationships

Every model element — a **PartDefinition**, a **RequirementUsage**, an **InterfaceUsage**, a **Connection** — is a JSON object with a stable UUID, a type (expressed as an **@type** field referencing the KerML/SysML v2 metamodel), and a set of properties. A part definition retrieved from the API looks like:

json

```
{
  "@type": "PartDefinition",
  "@id": "a3f8c2e1-7b44-4d2a-9c31-...",
  "name": "ThermalManagementSystem",
  "ownedFeature": [
    { "@id": "b12f..." },
    { "@id": "c34a..." }
  ],
  "declaredShortName": "TMS"
}
```

The **ownedFeature** array contains references by UUID to child elements — ports, properties, connections. Traversing the model means following these references through subsequent API calls. This is structurally identical to traversing a labeled property graph, which is the design intent.

Query operations use a structured query language defined in the specification:

http

POST /projects/{projectId}/commits/{commitId}/query-results

With a body like:

json

```
{
  "select": ["@id", "name", "@type"],
  "where": {
```

```
"@type": "RequirementUsage",  
"declaredName": { "$regex": ".*safety.*" }  
}  
}
```

This returns all requirement usages whose declared name contains "safety" — enabling a requirements management tool to pull a filtered view of the model's requirements without retrieving the entire model.

How interoperability actually works in practice

The interoperability promise is that any tool that implements the API client side can read from and write to any tool that implements the API server side. In the current ecosystem this means:

A systems engineer authors model content in Cameo Systems Modeler (which has v2 support in beta) or in SysIDE (the open-source VSCode extension). Both push commits to a conformant repository server — currently the reference implementation is a Java server maintained on GitHub by the SysML v2 team, with commercial alternatives from IntercaX (Synidea) and others emerging.

A simulation engineer's Python script queries the repository for part definitions, extracts interface port declarations and their quantity kinds and units, and automatically generates FMU connection topology — which FMU output maps to which FMU input — without the simulation engineer needing to open the modeling tool or read a document.

A requirements management system (DOORS Next, Polarion) pulls `RequirementUsage` elements from the repository and creates bidirectional traceability links — the requirement in DOORS is the same artifact as the `RequirementUsage` in the SysML v2 model, synchronized through the API rather than maintained as a separate copy.

A knowledge graph ingestion pipeline reads element commits as they arrive, extracts the JSON-LD representation, and loads elements and relationships into Neo4j or an RDF triplestore — creating a queryable semantic graph of the engineering model that Cypher or SPARQL queries can traverse.

A CI/CD pipeline checks out the model at every commit, runs conformance checks (are all requirements formally satisfied? are all ports typed with declared quantity kinds?), and fails the build if the model violates governance rules — exactly as a software CI pipeline fails on a failing test.

The version model — why commit semantics matter

The commit-based version model is what elevates the API above a simple CRUD interface. Consider a multi-contractor program where a prime contractor owns the top-level system architecture and three subsystem integrators each own a portion of the model:

Each integrator works in a branch — a named pointer to a sequence of commits diverging from a common parent. The prime contractor's integration team maintains the main branch. When an integrator's branch is ready to merge, the API supports retrieving the diff between the branch tip and

the merge target, so human reviewers can see exactly what changed before approving. After merge, every element has a commit-level audit trail: who changed it, when, and from what prior state.

This is configuration management for engineering models — something that SysML v1 programs attempted with tool-specific version management, ad hoc XMI export conventions, and document control systems that had no semantic understanding of the model content. The API makes model versioning a first-class, tool-neutral capability.

What is missing — the honest gaps

The specification is ahead of the implementation. As of 2024–2025, the conformant repository implementations are limited: the reference server works for research and prototyping, Syndeia has commercial support for some operations, and Cameo's v2 integration is in beta. The query language is specified but not uniformly implemented — some servers support only element retrieval by ID, not filtered queries. The merge and branch semantics are specified but the tooling for managing branches across multi-contractor programs is not yet productized.

The JSON representation is verbose. Traversing a large model through the API requires many sequential HTTP calls — following UUID references element by element — which is slow for large models over high-latency connections. Bulk export endpoints and efficient graph traversal queries are not yet standardized, which means every implementation invents its own performance optimization.

Perhaps most significantly, the API does not yet specify a notification or event subscription mechanism. A tool that wants to know when another tool has changed a model element it depends on cannot subscribe to changes — it must poll. For the live, reactive integration vision to be fully realized, a WebSocket or server-sent events extension is needed. This is on the roadmap but not in the current standard.

The knowledge graph and AI grounding connection

This is where the API's significance extends well beyond MBSE tooling interoperability. A SysML v2 repository, accessed through its API, is a semantically typed graph of engineering knowledge — every element is typed against the KerML metamodel, every relationship is named and directed, every property is declared with quantity kind and unit. This is precisely the structure that makes AI grounding tractable.

An AI assistant querying a SysML v2 repository can retrieve the actual declared interface between two subsystems — the port types, the quantity kinds, the connection constraints — rather than reasoning from training data about what such an interface might look like. The Circuit Breaker pattern applies directly: before an AI makes an architectural claim about the system, it grounds that claim against the repository. If the claim conflicts with what the model asserts, the discrepancy is detectable — and the MTBH metric becomes measurable because you have a formal, queryable ground truth to compare against.

The integration pattern that would fully realize this is one you have already architected in different domains: a federated knowledge graph where the SysML v2 repository elements are nodes, AADL component specifications are nodes, FMU model description variables are nodes, and PLM records are nodes — all connected by traversable, typed edges that make cross-domain queries possible without any single tool owning the complete picture. The SysML v2 API is the engineering community's first serious attempt to build the infrastructure that makes that pattern achievable at program scale rather than only in research prototypes

Co-simulation is the practice of running multiple simulation models simultaneously — each solving its own physics in its own tool — while exchanging state information at regular intervals so that the combined system behaves as an integrated whole. The "co" prefix captures the essential idea: models that were built independently, in different tools, using different mathematical approaches, cooperate to simulate a system that none of them could simulate alone.

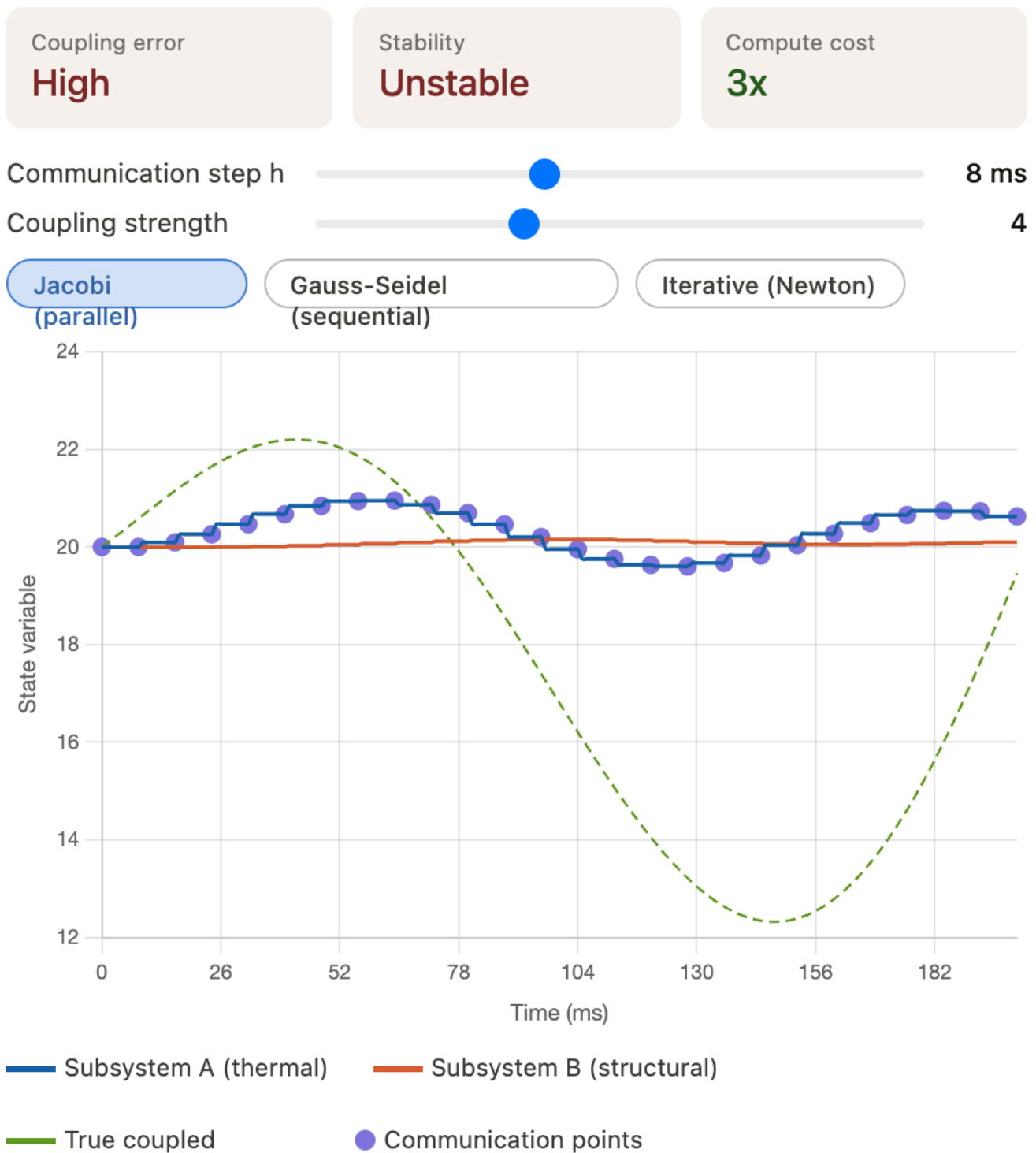
The need arises directly from physical reality. Real engineered systems do not respect domain boundaries. A jet engine is simultaneously a thermodynamic cycle, a structural stress field, a rotating mechanical system, a fuel flow network, and a control system. The heat generated by combustion affects material properties which affect structural behavior which affects rotor dynamics which affects aerodynamic efficiency which feeds back into combustion temperature. These interactions are not peripheral — they are often where the most important system-level behaviors emerge, including failure modes, performance limits, and control instability.

No single simulation tool captures all of this. CFD tools solve fluid dynamics. FEA tools solve structural mechanics. Modelica solves multi-domain lumped-parameter physics. Simulink solves signal flow and controls. Co-simulation is the architectural pattern that connects them.

The fundamental problem: time and coupling

When you run multiple simulators simultaneously, you face a problem that does not exist when running a single simulator: how do you synchronize them in time, and how do you handle the fact that each simulator needs inputs from the others that it has not yet computed?

A single physics solver advances time continuously, adjusting its step size based on the local dynamics — slowing down when things change rapidly, speeding up when the system is evolving slowly. When you connect two solvers, each advancing time at its own internal rate, you need a coordination layer that decides when information flows between them. That coordination layer is the master algorithm, and the moments when information exchanges happen are called communication points



The widget makes the core trade-off visceral. Drag the communication step slider to the right and watch the co-simulation trajectories diverge from the true coupled solution — the coupling error grows because each subsystem is operating on stale information from its partner for longer intervals. Increase coupling strength and the same step size that was stable becomes unstable,

because tightly coupled systems are more sensitive to the delay introduced at communication boundaries. Switch between schemes to see how iterative coupling dramatically improves accuracy at the cost of running each subsystem multiple times per communication step.

The three coupling schemes in depth

Jacobi coupling — sometimes called parallel or explicit coupling — is the simplest scheme. At each communication point, the master reads all outputs from all subsystems at the previous communication point, distributes them as inputs, and then advances all subsystems simultaneously through the next interval. The subsystems run in parallel, which is computationally attractive for large models on multi-core hardware. The cost is that every subsystem operates on information that is one communication step old. For weakly coupled systems with small step sizes this is negligible. For tightly coupled systems — where subsystem A's state strongly and rapidly influences subsystem B and vice versa — the one-step delay introduces an artificial phase lag that can destabilize the simulation even when the true coupled system is stable.

Gauss-Seidel coupling — sequential coupling — updates subsystems one at a time, passing each subsystem's fresh output immediately as input to the next. If the dependency ordering is $A \rightarrow B$ (A's state influences B more than the reverse), updating A first and then using A's fresh output to drive B reduces the error compared to Jacobi. The remaining error is the one-step delay on the $B \rightarrow A$ direction. This halves the coupling error compared to Jacobi for asymmetrically coupled systems and requires knowing (or discovering) the dependency ordering. For symmetric bidirectional coupling — thermal $\leftrightarrow$ structural, where heat causes deformation which changes thermal boundary conditions — neither direction is naturally "upstream" and Gauss-Seidel offers limited improvement over Jacobi.

Iterative coupling — Newton-Raphson or fixed-point iteration — treats the communication point as a nonlinear solve. The master advances all subsystems, collects their outputs, checks whether the inputs and outputs are mutually consistent (do the temperatures that A passed to B match the temperatures that B computed given what A told it?), and if not, repeats with corrected values until convergence. This essentially recovers the accuracy of a monolithic solver — the coupling error can be driven to near-zero — at the cost of running each subsystem k times per communication step, where k is the number of iterations needed for convergence. For strongly coupled systems with slow dynamics relative to the communication step, iterative coupling is the correct choice. For fast dynamics or real-time applications, the computational overhead may be prohibitive.

Multi-domain coupling in a real program — the aircraft ECS example

Environmental Control Systems in commercial aircraft are a canonical multi-domain co-simulation problem and one where the domain interactions are genuinely non-trivial. An ECS simultaneously involves:

Thermodynamics: bleed air extracted from the engine compressor stage expands through turbines, mixes with recirculated cabin air, and is distributed through ducts. The thermodynamic model — typically Modelica with the ThermoSysPro or Modelon Thermal Power libraries — tracks

temperature, pressure, and enthalpy at each node in the pneumatic network.

Structural mechanics: the duct walls and pressure vessel of the fuselage experience thermal expansion and pressure loading. At cruise altitude, the pressure differential between cabin (8,000 ft equivalent) and outside (40,000 ft) creates significant hoop stress in the fuselage skin. Temperature gradients from cold soak on the ground to hot bleed air flow at cruise create thermal cycling that drives fatigue. The structural model — typically Nastran or ANSYS — computes these stresses and the resulting deformations, which feed back into duct cross-sections and flow resistance.

Controls: the pack flow control valve, the mix manifold valve, and the temperature control valve are driven by a control system that reads cabin temperature sensors and pack outlet temperatures, computes valve commands through PID or model-predictive control logic, and sends actuator commands. The controls model — typically Simulink — needs the thermodynamic state as input and produces valve position commands as output, which change the boundary conditions of the thermodynamic model.

Engine offtake: bleed air extraction creates a parasitic load on the engine that affects specific fuel consumption. The propulsion model — possibly a Numerical Propulsion System Simulation (NPSS) model or a Modelica turbofan model — needs to know how much bleed flow is being extracted at what pressure, and returns the resulting thrust penalty and fuel burn rate change.

None of these can be simulated accurately in isolation. The control system tuning depends on the thermodynamic response characteristics. The thermodynamic behavior depends on duct geometry which depends on structural deformation under load. The fuel burn impact of ECS depends on engine operating point which depends on aircraft weight which changes as fuel is burned. Co-simulation connects them all.

What the master algorithm must manage

Beyond step coordination, a production master algorithm handles several problems that toy examples elide:

Algebraic loops occur when subsystem A's output at time t depends on subsystem B's output at time t , which simultaneously depends on A's output at time t . This is a zero-delay circular dependency that cannot be resolved by any sequential update ordering — it requires iteration or the introduction of an artificial small delay (which introduces error). The master must detect algebraic loops at model assembly time and flag them for the engineer to resolve, either by breaking the loop with a unit delay, by restructuring the subsystem boundaries, or by using iterative coupling.

Event handling is needed when subsystem state changes discontinuously — a valve snaps open, a contact makes or breaks, a relay trips, a fluid flashes from liquid to vapor. At these events the coupled equation system changes structure, and the master must detect the event time, halt all subsystems at that instant, reinitialize state consistently across subsystems, and resume. Missing events — advancing past them without detection — produces silently wrong trajectories. Good masters use event localization algorithms (bisection on the event indicator function) to find event times precisely.

Step size adaptation balances accuracy against computation time. A master that monitors the rate of change of exchanged variables can shrink the communication step when dynamics are fast (during a valve transient, a thermal shock, a control mode switch) and expand it when the system is evolving slowly (steady cruise, thermal soak). This requires subsystems that can accept variable step requests — not all FMU implementations support this robustly.

Error estimation across subsystems is the hardest problem and largely unsolved in production tools. Within a single solver, error estimators compare the solution at different step sizes to bound local truncation error. Across subsystems communicating through a master, the coupling error — the error introduced by using stale inputs — is not easily estimated without running the simulation at multiple step sizes and comparing, which doubles the computation cost. Most practitioners use engineering judgment: run at a step size that feels conservative, validate against monolithic simulations or physical test data, and build in safety margins.

The MBSE integration — how co-simulation connects to the model

The connection to SysML v2 is architectural. The SysML v2 model declares the system's components and their interfaces — port types, quantity kinds, units, connection topology. An FMI co-simulation setup is a physical instantiation of that interface declaration: each SysML part becomes an FMU, each SysML port becomes an FMU variable declared in the model description XML, each SysML connection becomes a master algorithm coupling specification.

The semantic alignment between the SysML v2 model and the FMI co-simulation configuration is currently achieved manually in most programs — a systems engineer reads the architecture model and configures the simulation by hand. The opportunity — and it connects directly to the knowledge graph work you've done — is to make this automatic. If the SysML v2 repository is queryable through its REST API, and the FMU model description XMLs are loaded into the same graph, then the coupling configuration can be derived: find all SysML connections between ports, find the FMU variables that correspond to those ports, verify unit compatibility, and generate the master algorithm configuration. Interface mismatches — a thermal port connected to a mechanical port, a temperature variable coupled to a pressure variable — become detectable at model assembly time rather than at simulation runtime or, worse, in post-processing when results look suspicious.

This is the Circuit Breaker pattern applied to simulation setup. The grounding layer is the semantic graph that connects architectural intent (the SysML model) to simulation implementation (the FMU variables), and the breaker trips when the simulation configuration violates the architectural specification — before any computation runs and before any engineer has to read a confusing error message from a solver that received out-of-range inputs because two subsystems were connected in the wrong direction

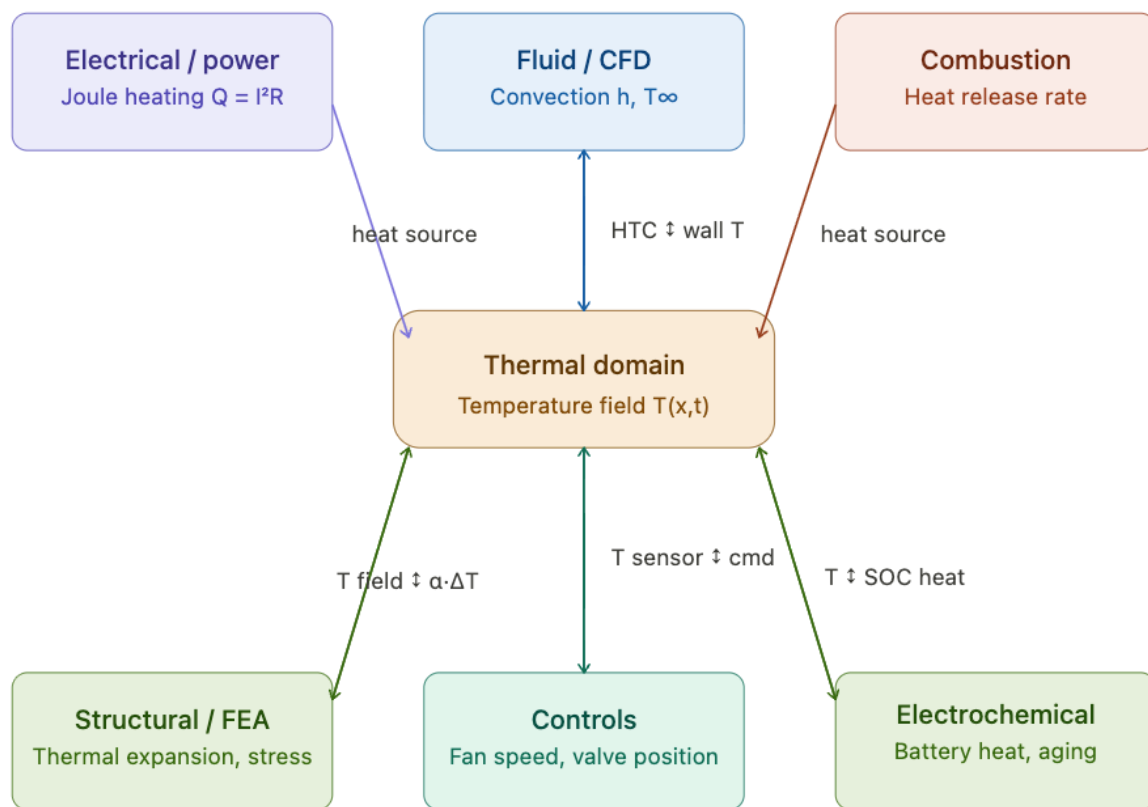
Thermal simulation sits in an interesting position within the MBSE stack — it is one of the oldest and most mature physics domains in engineering simulation, yet its integration into model-based workflows remains one of the most manually intensive handoffs in practice. Understanding why

requires looking at both the physics and the data flow.

The fundamental challenge: thermal is always coupled

Pure structural analysis can, with some effort, be treated in isolation — compute loads, apply boundary conditions, get stresses. Thermal analysis almost never can. Heat generation depends on electrical dissipation (power electronics, batteries, motors), friction (bearings, brakes, seals), and chemical reactions (combustion, electrochemistry). Heat removal depends on fluid flow (coolant circuits, airflow over surfaces, phase-change refrigerants). The resulting temperature field affects structural stiffness and thermal expansion (thermo-mechanical coupling), material properties and reaction rates (thermo-chemical coupling), and electrical resistance and semiconductor behavior (thermo-electrical coupling). Thermal is the coupling domain — it receives energy from almost every other physics and returns boundary conditions to almost every other physics.

This coupling is what makes thermal integration in MBSE architecturally significant. A thermal model that lives in isolation from the rest of the system model produces answers that are correct for the boundary conditions assumed but potentially wrong for the actual system operating conditions, which are determined by the interaction of all domains simultaneously



Click any domain to explore the coupling interface ↗

The thermal simulation tool landscape and what each does

Thermal simulation is not a monolithic field — different tools address different regimes of the physics, and the MBSE integration challenge differs depending on which tool sits in the thermal role.

Lumped-parameter network tools (Siemens Thermal Desktop / NX Thermal, ESATAN-TMS for spacecraft, Modelica's Thermal library) model the system as a network of thermal capacitances connected by thermal resistances, convection coefficients, and radiation exchange factors. A spacecraft thermal model in ESATAN-TMS might have thousands of nodes representing surface panels, electronics boxes, and structural members, connected by conductances calculated from geometry and material properties. The governing equations are ordinary differential equations (ODEs) — one energy balance per node — which are numerically cheap to solve and easy to couple through FMI because the interface is simply a set of temperatures and heat flows at boundary nodes.

Finite Element Analysis tools (ANSYS Thermal, Abaqus/Standard, MSC Nastran SOL 153/159) solve the heat conduction PDE across a discretized solid mesh. These are the tools of choice when spatial temperature gradients within a component matter — junction temperatures in power electronics, thermal gradients in turbine blades, heat spreading in printed circuit boards. FEA thermal models are spatially rich but computationally expensive, which creates the ROM problem discussed in the FMI context: direct integration into co-simulation requires reduced-order model export.

Computational fluid dynamics tools with conjugate heat transfer (ANSYS Fluent, Siemens Star-CCM+, OpenFOAM) solve the coupled fluid flow and heat transfer problem simultaneously — the temperature field in the solid and the temperature and velocity field in the adjacent fluid are solved together, with the interface between them enforcing heat flux continuity. This is the highest-fidelity approach for convective cooling problems (electronics enclosures, heat exchangers, cooled turbine blades) but is computationally expensive to the point where full transient conjugate simulations are often impractical for system-level integration. The typical pattern is to run CHT offline to generate heat transfer coefficient (HTC) correlations as a function of operating conditions, then use those correlations as boundary conditions in the faster lumped-parameter or FEA thermal model.

1D thermal-fluid network tools (Flownex, Flowmaster, Modelica Fluid) sit between lumped-parameter and full CFD. They solve 1D conservation equations along flow paths — mass, momentum, and energy — capturing pressure drop, flow distribution, and convective heat transfer in piping networks, heat exchangers, and cooling circuits. These are the tools for designing ECS ducting, engine oil cooling circuits, battery thermal management coolant loops, and building HVAC systems. Their computational cost is orders of magnitude below CFD and they couple naturally through FMI.

How thermal models enter the MBSE workflow — the four integration points

At the requirements level, thermal requirements flow from the SysML model into the thermal simulation as boundary conditions and acceptance criteria. A SysML **RequirementUsage** like "the power module junction temperature shall not exceed 150°C under any operating condition" defines the thermal simulation's pass/fail criterion. In SysML v2 with formal constraint expressions, this requirement can carry a machine-readable form that an analysis tool can evaluate directly against simulation output — closing the loop between the architectural requirement and the simulation result without a human reading both and making the comparison manually.

At the interface definition level, the SysML internal block diagram (IBD in v1) or port declarations (v2) defines what thermal quantities flow between subsystems. A **ThermalPort** with declared **TemperatureValue** and **HeatFlowRateValue** variables is the architectural specification that the co-simulation coupling must implement. If the SysML model declares that the battery pack exchanges temperature and heat flow rate with the thermal management system, the FMU coupling configuration should reflect exactly those variables — and a query against the SysML v2 API combined with the FMU model description XMLs can verify it does.

At the subsystem physics level, the thermal simulation itself runs. This is where tool choice, mesh fidelity, boundary condition sourcing, and solver configuration live. The outputs — temperature fields, heat flux distributions, maximum temperatures, thermal resistance maps — are the simulation's primary deliverables.

At the system integration level, thermal simulation outputs feed back into other analyses. Structural FEA receives the temperature field as a load case for thermal stress analysis. Control system design receives the thermal time constants (how quickly does junction temperature rise at a given power dissipation rate?) as plant model parameters. Reliability analysis receives the junction temperature distribution as input to Arrhenius-based lifetime models. Pareto optimization receives maximum component temperature and total cooling power consumption as objective function values.

The thermo-mechanical coupling workflow in detail

Thermo-mechanical coupling — computing the structural stresses and deformations that result from non-uniform temperature distributions — is the most common cross-domain thermal integration in aerospace and defense programs, and it illustrates the workflow challenges clearly.

The one-way (sequential) approach is the most common in practice: run the thermal analysis first, export the nodal temperature field, import it into the structural model as a body load, run the stress analysis. This works when the deformation does not significantly change the thermal boundary conditions — when thermal expansion does not alter flow paths, when deformed geometry does not significantly change radiation view factors, when material property changes from temperature are modest. For many practical cases this is adequate and it keeps the two analyses decoupled, which is computationally tractable and organizationally convenient.

The two-way (iterative) approach is needed when deformations do matter. A turbine blade's thermal expansion changes the blade tip clearance, which changes the leakage flow, which changes the

convective boundary condition on the blade tip, which changes the temperature field, which changes the deformation. Solving this requires running thermal and structural analyses in a coupled loop until both converge to a mutually consistent solution. In practice this means either tight co-simulation through FMI (the thermal FEA and structural FEA each export an FMU, a master algorithm orchestrates the exchange of temperature fields and deformed geometry) or using a coupled analysis within a single tool that handles both physics (ANSYS Mechanical with coupled-field elements, Abaqus with sequentially coupled thermal-stress analysis).

The mesh mapping problem is a practical complication that receives less attention in academic treatments than it deserves. The thermal FEA mesh and the structural FEA mesh are almost never identical — they are optimized for different physics. Thermal meshes need refinement at heat sources and heat transfer surfaces; structural meshes need refinement at stress concentrations and geometric features. Transferring a temperature field from a 50,000-node thermal mesh onto a 200,000-node structural mesh requires interpolation, and the interpolation must be conservative — the integrated heat content of the mapped field must equal that of the original. Tools like ANSYS System Coupling and MpCCI (Mesh-based parallel Code Coupling Interface) handle this mapping. Getting it right matters: a non-conservative temperature interpolation produces artificial thermal stresses that do not correspond to any real physics.

Reduced-order models and why they are necessary

A full conjugate heat transfer CFD model of an electronics enclosure might have 10 million cells and take 8 hours to run on a 32-core cluster for a single steady-state operating point. This is entirely incompatible with system-level co-simulation, where hundreds of time steps need to be computed at each operating condition, and design optimization that sweeps thousands of parameter combinations.

The solution is the reduced-order model (ROM): a low-dimensional surrogate that captures the dominant thermal behavior with a fraction of the computational cost. Several approaches exist with different fidelity-cost trade-offs:

Compact thermal models replace a detailed component (an electronics package, a heat sink) with a small network of resistances and capacitances that reproduce the component's external thermal behavior — junction-to-case resistance, case-to-ambient resistance, thermal capacitance — without modeling the internal temperature distribution. These are the standard models used in system-level electronics thermal simulation tools like Mentor Flotherm or ANSYS Icepak.

Proper Orthogonal Decomposition (POD) ROMs extract the dominant spatial modes of the full-order temperature field from a set of offline simulations run at different operating conditions, then represent any new condition as a linear combination of those modes with coefficients solved by a small system of equations. A 10-million-cell CFD model might reduce to a 50-mode ROM that runs in milliseconds while reproducing temperature distributions at new operating points with errors below 1%. This is what gets exported as an FMU for system-level integration.

Data-driven thermal surrogates — Gaussian Process regression, neural networks trained on

simulation outputs — function similarly to RSM surrogates. They are fast to evaluate once trained but provide no physical insight and can fail outside the training envelope without warning. Their advantage over POD ROMs is that they do not require access to the full-order solver's internal state — you can build them from input-output simulation data alone.

The MBSE knowledge graph connection

The recurring challenge in thermal integration is traceability — knowing which thermal analysis result corresponds to which system operating condition, which requirement it validates, which simulation tool produced it with which mesh and which boundary conditions. In large programs this traceability is maintained in documents and spreadsheets, which means it is always partially out of date and requires human effort to navigate.

The knowledge graph pattern you developed at Verizon and instantiated in ARCH at AbbVie applies directly. A thermal analysis artifact — a temperature field result, a maximum junction temperature, a thermal resistance map — is a node in the knowledge graph. It has typed relationships to the simulation tool that produced it, the mesh file used, the boundary conditions applied, the system operating condition it represents, the SysML v2 requirement it validates, and the co-simulation FMU it feeds into. Querying "which thermal analyses support the junction temperature requirement for the power module under worst-case ambient conditions" becomes a graph traversal rather than a document search.

This is where the Circuit Breaker pattern for AI grounding becomes particularly valuable in engineering workflows. An AI assistant helping a thermal engineer set up a new analysis can query the knowledge graph to retrieve the applicable thermal requirements from the SysML v2 repository, the boundary conditions from the co-simulation topology, the previous analysis results for comparison, and the approved mesh generation practices from the organization's simulation standards — grounding every recommendation against verified artifacts rather than generating plausible-sounding boundary conditions from training data alone. The Mean Time Between Hallucinations metric becomes directly applicable: how often does the AI recommend a boundary condition that the knowledge graph can verify is inconsistent with the system-level thermal requirements or the co-simulation interface specification?

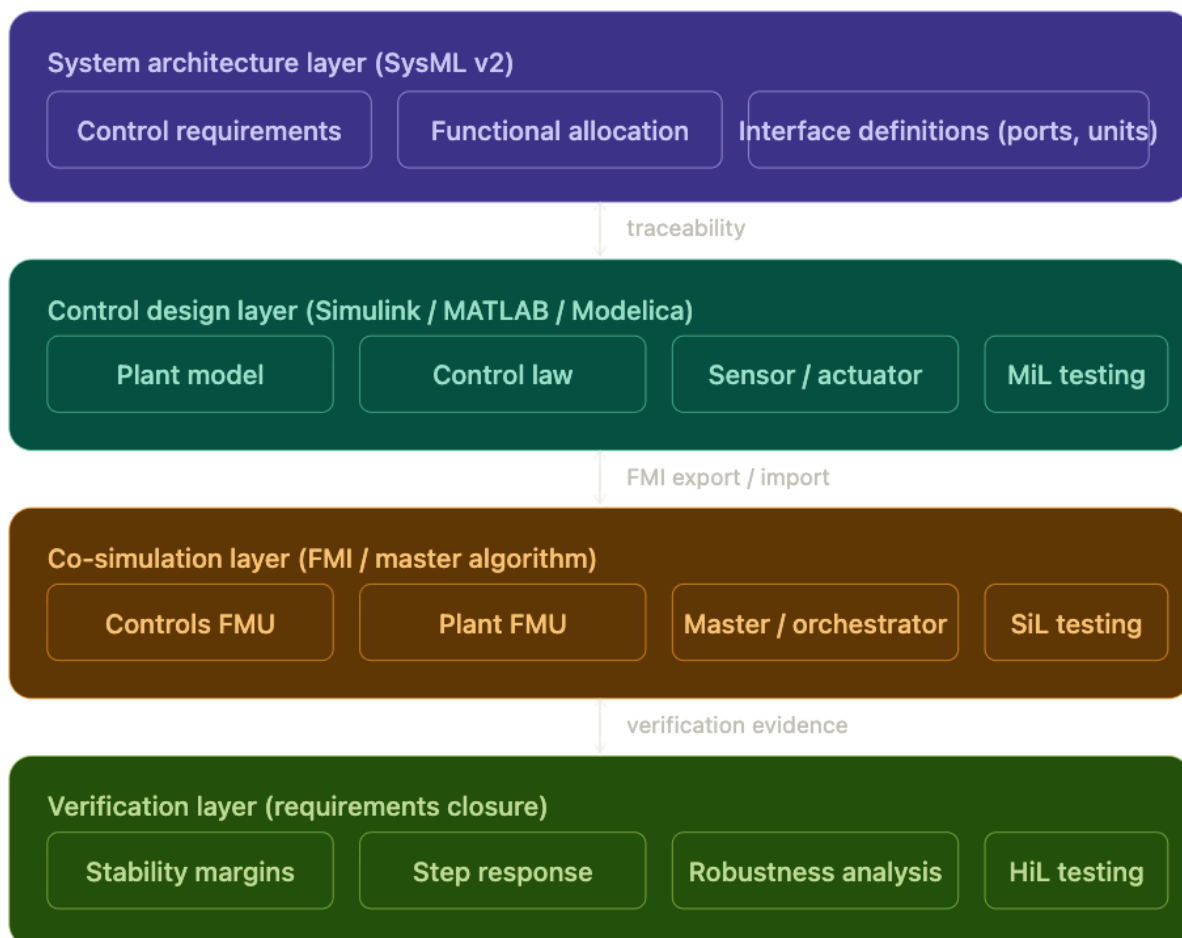
Controls simulation and system architecture modeling have historically operated in parallel universes that only intersect at program reviews — the systems engineer draws the architecture, the controls engineer builds the Simulink model, and the connection between them is a document describing requirements that may or may not accurately reflect what either model actually implements. Closing that gap is one of the central value propositions of MBSE, and it is harder than it looks because controls and architecture modeling operate at fundamentally different levels of abstraction.

Why the gap exists — the abstraction mismatch

A system architecture model in SysML describes *what* the system is: components, interfaces, requirements, allocations, behavioral specifications at a functional level. A controls simulation in

Simulink describes *how* a specific control law behaves when connected to a specific plant model under specific operating conditions. The architecture model is declarative and multi-stakeholder; the controls model is computational and specialist. They use different mathematical languages, different tooling paradigms, different update cadences, and different validation criteria.

The gap is not accidental — it reflects a genuine difference in what each artifact is for. The challenge is building the traceability and data flow connections that allow changes in one to propagate appropriately to the other, and that allow verification evidence generated in the controls simulation to close against requirements expressed in the architecture model.



Click any element to explore further ↗

Layer 1: How control requirements live in the architecture model

The starting point is expressing control system requirements in a form that is both human-readable for stakeholders and machine-processable for verification. In SysML v1 this was done with `<<requirement>>` stereotypes and text — readable, but not evaluable. In SysML v2 with formal constraint syntax, a control requirement can carry a mathematical condition:

```
requirement stabilityMarginReq {  
  subject cs : FlightControlSystem;  
  require constraint {  
    cs.gainMargin >= 6.0 [dB] and  
    cs.phaseMargin >= 30.0 [deg]  
  }  
  doc /* Derived from MIL-HDBK-1797B stability requirements */  
}
```

This is not just documentation — it is an assertion that a conformant analysis tool can evaluate by querying the simulation results and checking whether the computed gain and phase margins satisfy the constraints. When the Simulink model completes a frequency response analysis and reports gain margin and phase margin values, those results can be compared against this requirement automatically, generating verification evidence that is traceable to the architectural requirement rather than to a document that someone read.

Control system interface declarations in SysML v2 specify the physical quantities flowing between the controller and the plant. A flight control surface actuator interface might declare:

```
port def ActuatorCommandPort {  
  out attribute deflectionCmd : AngleValue;  
  out attribute rateCmd : AngularRateValue;  
}  
  
port def ActuatorFeedbackPort {  
  in attribute deflectionMeas : AngleValue;  
  in attribute loadMeas : ForceValue;  
}
```

These port declarations with typed quantity kinds and ISO 80000 units are the architectural contract. The Simulink model implements this contract — its input and output signal dimensions and units must match. Connecting the two requires a mapping layer, which is where most of the practical integration work lives.

Layer 2: The plant model and its origin

The plant model in a controls simulation is the mathematical representation of the physical system being controlled — what the controller acts on. How this plant model is built, and how accurately it represents the real system, determines whether the control law designed in simulation will work on the real hardware.

In early-phase design the plant model is often a linearized analytical model — transfer functions derived from first principles, or state-space matrices from simplified physics. A battery management system's plant might start as a first-order RC circuit model of the battery's voltage-current behavior, which is tractable for control design but misses temperature dependence, aging effects, and internal short circuit dynamics. These simplifications are conscious engineering choices, not errors — the linearized plant is sufficient for initial control law structure selection and gain calculation.

As design matures, the plant model fidelity increases. A Modelica model of the battery electrochemistry and thermal behavior — validated against cell-level test data — replaces the RC circuit. The Modelica model exports as an FMU. The Simulink controls model, which was previously driving an analytic plant, now drives the Modelica FMU through the FMI co-simulation interface. The control law that worked against the simple plant may need retuning against the higher-fidelity plant — or it may behave differently in ways that reveal stability margins were inadequate when the simplified plant masked certain dynamics.

This plant model evolution is the core validation risk in controls-MBSE integration. The architecture model needs to track which plant model fidelity level was used to validate each control requirement — a control law designed against a linearized plant and a control law validated against a full nonlinear multi-physics FMU provide fundamentally different levels of verification confidence. The knowledge graph, if populated correctly, preserves this traceability: requirement → verification analysis → simulation configuration → plant model fidelity level → validation test data.

The Simulink-SysML connection — the practical mechanics

The integration between Simulink and SysML architecture models happens through several mechanisms that have different trade-off profiles.

Manual interface specification is the baseline approach used in most programs today. A systems engineer defines the control interface in SysML — signal names, directions, quantity kinds, units, timing requirements. A controls engineer implements those signals as Simulink bus objects and validates conformance by inspection. This works, is universally supported, and is fragile — interface changes in the SysML model do not automatically propagate to the Simulink model, and the two diverge over time.

Syndeia and similar integration tools provide bidirectional synchronization between SysML tools (Cameo, Rhapsody) and Simulink. A Syndeia adapter reads the SysML interface definition, generates Simulink bus type definitions conformant to that interface, and can check an existing Simulink model for conformance. When the architecture model changes — an additional feedback signal added, a unit corrected from meters per second to knots — Syndeia can propagate that change to the Simulink interface definition and flag any blocks that need updating. This is not automatic code generation; it is interface synchronization that reduces the drift between models without eliminating the need for human judgment about what the changes mean.

Automatic code and model generation from SysML behavioral models is the more ambitious approach, attempted through tools like IBM Rhapsody's code generator and Simulink's own

model-based design workflow. A SysML state machine or activity diagram at the architecture level is refined into a Simulink block diagram that implements it. This works well for control mode logic — the supervisory state machine that decides which control law is active under which conditions — and less well for the continuous-time dynamics of the control law itself, which are better designed natively in Simulink using control-theoretic tools (root locus, Bode plots, LQR optimization) that have no equivalent in SysML.

SysML v2 API-driven integration is the emerging approach. A Python or Julia script queries the SysML v2 repository for interface definitions — port types, quantity kinds, units, timing constraints — and uses that information to configure or validate the Simulink model's interface. The script can also push verification results back to the repository — tagging requirement elements with simulation results and pass/fail status. This approach requires no proprietary adapter tool, is scriptable and automatable, and can run as part of a CI pipeline. Its limitation is that it requires writing and maintaining the integration scripts, which is a software development activity that systems and controls engineers may not be resourced or inclined to do.

Model-in-the-loop to hardware-in-the-loop: the verification progression

Controls verification in MBSE follows a structured progression through simulation environments of increasing fidelity and decreasing speed, each addressing a different class of requirement.

Model-in-the-loop (MiL) testing runs the control law in Simulink connected to a plant model, also in Simulink, with no hardware involved. This is the environment for control law design — tuning gains, optimizing performance against the linearized plant, verifying basic stability and response time. MiL testing is fast (faster than real-time), fully observable (every internal signal is accessible), and easily scriptable for systematic parameter sweeps. Verification evidence from MiL testing addresses requirements about control law behavior under idealized conditions — the nominal design point.

Software-in-the-loop (SiL) testing compiles the control law code that will run on the embedded processor and runs it in simulation on a PC, still connected to a plant model. The key difference from MiL is that the actual compiled code is exercised, not just the Simulink block diagram representation. Fixed-point arithmetic effects, numerical precision limitations of the embedded processor, and code generation artifacts are all captured in SiL testing. A control law that was stable in MiL can exhibit limit cycling or instability in SiL due to quantization effects that the floating-point Simulink simulation did not expose.

Processor-in-the-loop (PiL) testing runs the compiled code on the actual target processor — the flight computer, the ECU, the battery management system controller — while the plant model still runs in simulation on a separate PC. This reveals processor-specific timing behavior, interrupt handling latency, and real-time operating system scheduling effects. A control loop that assumes 10ms execution time but actually takes 12ms on the real processor due to cache effects will exhibit phase lag that degrades stability margins.

Hardware-in-the-loop (HiL) testing connects the real controller hardware to a real-time plant simulation running on a dedicated HiL platform (dSPACE, National Instruments VeriStand,

Speedgoat). The plant simulation runs at real-time — fast enough that the controller cannot distinguish it from the actual physical plant — while the controller experiences realistic sensor signals and actuator loads. HiL testing addresses requirements about closed-loop system behavior including all hardware timing effects, sensor noise, actuator nonlinearities, and communication bus latency. It is the highest-confidence simulation-based verification step before actual physical integration testing.

Each of these steps generates verification evidence that needs to be traced back to specific requirements in the architecture model. In most programs today this traceability is maintained in test report documents. In a mature MBSE knowledge graph this traceability is a first-class graph relationship: the HiL test run at operating condition X with plant model version Y produced gain margin Z, which satisfies requirement R at confidence level C, where the confidence level reflects the fidelity of the plant model used and the coverage of operating conditions tested.

Control architecture patterns and their MBSE representation

Different control architectures create different integration challenges with the MBSE model, and recognizing these patterns helps clarify what needs to be represented architecturally versus what belongs in the controls simulation.

Mode logic — the supervisory state machine that selects which control law is active — belongs in the architecture model as well as the controls simulation. A flight control computer's mode transitions (normal → direct law → mechanical backup in an Airbus fly-by-wire system, for example) are safety-critical behavioral requirements that certification authorities need to understand at the system level. These are best represented in SysML state machines at the architecture level, then refined into Simulink Stateflow charts that implement the same logic in executable form. The architecture-level state machine serves as the requirements specification; the Stateflow chart serves as the implementation. Keeping them synchronized — so that a mode transition added to the architecture is reflected in the Stateflow implementation — is the integration maintenance challenge.

Gain scheduling — varying control law parameters as a function of operating point — creates a parameter management challenge that sits at the architecture-controls interface. The scheduling variables (Mach number, altitude, airspeed, battery state of charge) are system-level quantities whose values come from sensors that are part of the system architecture. The gain tables are implementation details of the control law. The MBSE integration challenge is ensuring that the scheduling variable definitions in the architecture (what signal, what units, what valid range, what update rate) are consistent with how those variables are used in the control law — a mismatch in units between the architecture's angle-of-attack definition and the gain scheduler's input is exactly the kind of interface error that a formally typed SysML v2 port declaration connected to the Simulink signal definition would catch automatically.

Fault detection and reconfiguration logic — detecting sensor failures, actuator failures, and plant anomalies, and reconfiguring the control law in response — spans both the architecture model and the controls simulation in ways that are particularly difficult to integrate. The fault modes are defined

at the architecture level (AADL Error Model Annex, SysML failure mode allocations, fault trees). The detection logic and reconfiguration algorithms are implemented in the controls simulation. The connection between a declared failure mode in the architecture and the detection threshold implemented in the controller is a traceability link that, if not maintained, allows the architecture and implementation to drift in ways that only manifest as missed detections or false alarms in late-phase testing.

The connection to the cross-domain lattice of ontologies

The controls-MBSE integration challenge is a specific instance of the general knowledge federation problem you have addressed in multiple domains. The architecture model and the controls simulation are two authoritative sources that need to remain consistent without either one subordinating the other — exactly the multi-authoritative-source problem that semantic governance frameworks address.

The Circuit Breaker pattern applies precisely here. An AI assistant helping a controls engineer configure a Simulink model can query the SysML v2 repository for the declared interface — signal names, types, units, timing constraints — and validate the Simulink model's interface against that declaration before any simulation runs. A discrepancy — a signal declared as `AngleValue` in the architecture but wired to a block that expects a dimensionless fraction in Simulink — trips the breaker and surfaces the mismatch for human resolution rather than allowing it to propagate silently into a simulation result that passes numerically but validates the wrong behavior.

The MTBH metric in this context is measurable as the rate at which controls simulation configurations that were not grounded against the architecture model produce verification results that later prove inconsistent with system-level requirements — the simulation said the control law meets the stability margin requirement, but the requirement it checked against was stale, because the architecture model had been updated to tighten the margin and the verification script was not re-run. This is the dominant failure mode in controls-MBSE integration today, and it is exactly the class of failure that a live, queryable, version-aware knowledge graph connection between the SysML v2 repository and the verification evidence store is designed to prevent.

Hardware-in-the-loop (HiL) simulation is the practice of replacing part of a real system with a real-time simulation while leaving the rest as actual physical hardware — creating a closed-loop test environment where real hardware believes it is operating in its actual system context, even though most of that context is computed rather than physical. The "loop" is literal: sensor signals flow from the simulation to the hardware, the hardware processes them and generates actuator commands, those commands flow back into the simulation, which updates its plant state accordingly, all within timing tight enough that the hardware cannot distinguish simulation from reality.

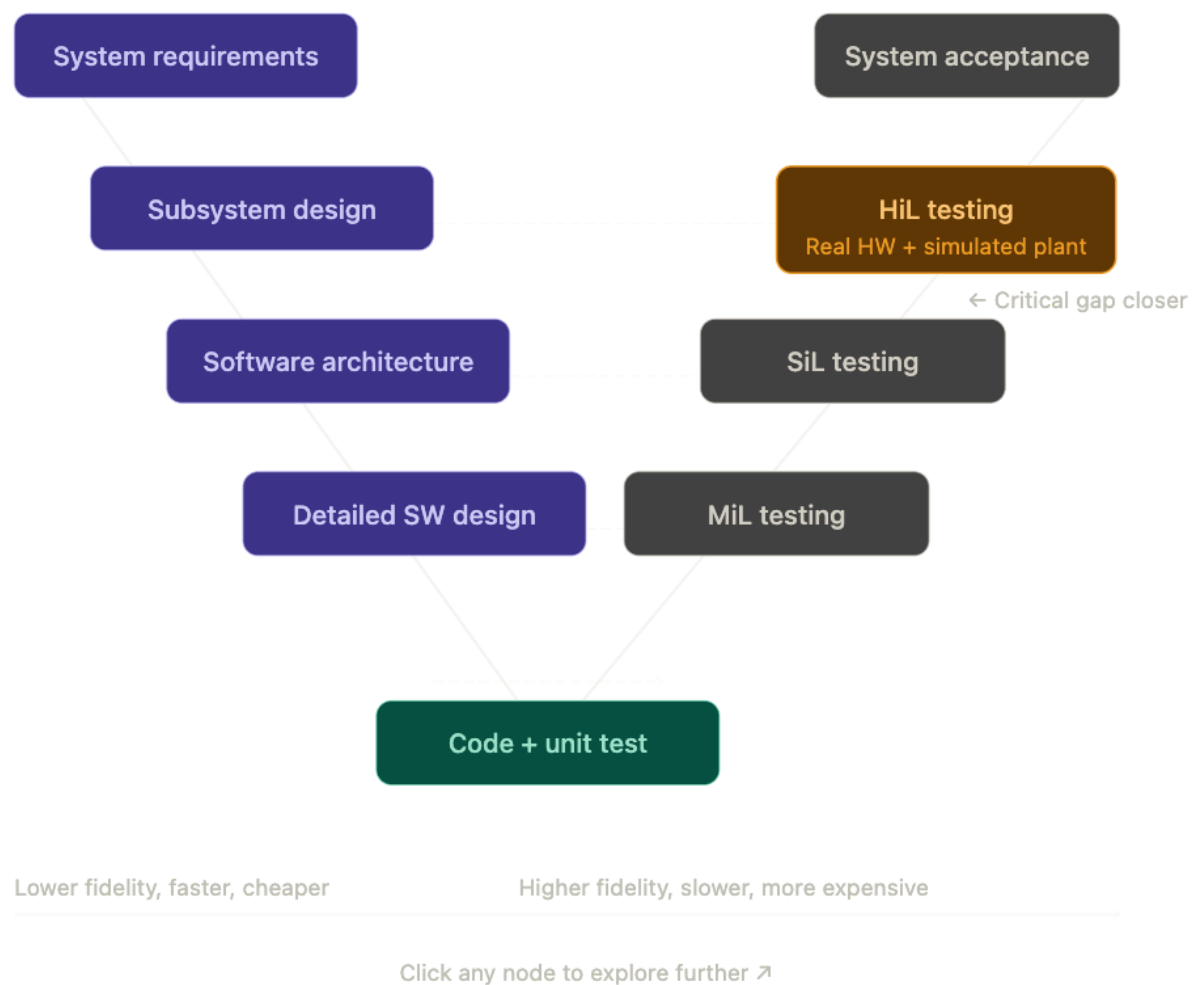
The word "real-time" carries precise meaning here. The simulation must complete each computational cycle within the same wall-clock time as the physical system's control cycle — typically 1ms to 10ms for automotive and aerospace applications, sometimes microseconds for

power electronics. If the simulation runs slower than real-time, the hardware receives delayed sensor updates and the closed loop breaks down. This real-time constraint is what separates HiL from the higher-fidelity but non-real-time simulations that precede it in the verification progression.

Why HiL exists — the fundamental verification gap

Model-in-the-loop and software-in-the-loop testing verify the control algorithm's behavior in simulation. But between the control algorithm running correctly in simulation and the control algorithm running correctly on real hardware embedded in a real system lies a class of failures that simulation cannot reveal: timing effects of the actual processor and operating system, electrical characteristics of the actual I/O hardware, communication bus behavior under real loading, power supply transients affecting processor behavior, EMI susceptibility of the actual PCB, and the nonlinear behavior of actual sensors and actuators that differ from their idealized simulation models.

HiL testing closes this gap by making the hardware exercise its actual embedded software in a realistic dynamic environment without requiring the full physical system to exist. For a commercial aircraft flight control computer, the alternative to HiL would be to test the real controller against a real aircraft — which is prohibitively expensive, potentially dangerous in early development phases, and provides poor observability since you cannot inject arbitrary failure modes into a flying aircraft



The physical architecture of a HiL system

A HiL rig consists of four physical layers that work together to create the illusion of a real operating environment for the unit under test.

The *real-time target* is a dedicated computer — not a standard PC — running a deterministic real-time operating system. dSPACE systems use their own SCALEXIO or DS1007 hardware running dSPACE's real-time kernel. National Instruments uses CompactRIO or PXI chassis running NI-Linux Real-Time or LabVIEW Real-Time. Speedgoat's Performance real-time target runs MATLAB/Simulink models compiled to bare-metal code using Simulink Real-Time. What makes these different from an ordinary computer is determinism: the simulation step executes in exactly the same wall-clock time every cycle, with no jitter from operating system scheduling, memory management, or background tasks. This determinism is what allows the hardware under test to trust the signals it receives.

The *I/O interface* connects the real-time target electrically to the unit under test. This is not a

software interface — it is a rack of signal conditioning hardware: digital-to-analog converters that generate analog voltage signals representing sensor outputs, analog-to-digital converters that read actuator command voltages, digital I/O for discrete signals, CAN bus interfaces, ARINC 429 or 664 avionics bus interfaces, PWM generators, and sometimes custom interface hardware for specific signal types. The electrical characteristics of this interface must match what the unit under test expects — voltage ranges, impedance, rise times, noise levels. A flight control computer expecting a 0-5V analog signal from an angle sensor receives exactly that from the D/A converter, and cannot distinguish it from a real sensor.

The *unit under test (UUT)* is the real hardware — typically an electronic control unit (ECU), a flight control computer (FCC), a battery management system (BMS), or a domain controller. It runs its actual production software, in its actual enclosure, powered by its actual power supply. It does not know it is connected to a simulation. It processes its simulated sensor inputs, executes its control algorithms, and drives its actuator outputs exactly as it would in the vehicle or aircraft.

The *plant simulation* is a real-time mathematical model of everything the UUT is not — the engine, the vehicle dynamics, the aircraft aerodynamics, the battery chemistry, the hydraulic actuators, the thermal environment. This model runs on the real-time target, updating at the simulation step rate, and is the simulation component that benefits most from the FMI and co-simulation infrastructure discussed earlier. A high-fidelity plant model, perhaps assembled from multiple FMUs representing different physics domains, gives the HiL test the dynamic realism needed to exercise edge cases that matter.

What HiL tests that earlier simulation stages cannot

The unique value of HiL relative to MiL and SiL testing comes from several hardware-specific phenomena that simulation cannot replicate.

Real-time execution effects are the primary motivation for HiL over SiL. The production software running on the actual processor exhibits timing behavior — cache effects, interrupt latency, RTOS task switching overhead, DMA transfer timing — that compiled simulation on a PC does not reproduce. A control loop designed for 10ms execution at 100Hz may actually execute in 10.3ms on average on the real ECU due to branch misprediction in a critical computation path, with occasional 12ms spikes when the CAN interrupt fires during the computation. These 20% timing violations introduce phase lag in the control loop that degrades stability margins below what SiL testing showed. HiL reveals this; SiL does not.

I/O hardware behavior is the second class of unique HiL findings. Real sensors have electrical characteristics — output impedance, capacitance, noise spectral density, offset drift with temperature — that differ from their simulation models. Real actuator drivers have switching transients, back-EMF spikes, and current limiting behaviors that affect the control loop when the actuator load changes rapidly. The signal conditioning hardware in the UUT — analog filters, sample-and-hold circuits, A/D converters — introduces phase lag and quantization that affects closed-loop stability. HiL exercises all of this through real electrical connections; SiL exercises none of it.

Communication bus loading under realistic conditions is a third class. A CAN bus with 20 nodes all transmitting at their scheduled rates creates arbitration contention that affects message delivery timing. An ECU that assumes its sensor messages arrive at exactly their nominal 5ms period will behave differently when those messages experience 0.3ms of bus arbitration delay under peak loading. HiL with a realistic bus simulation — or better, with real bus nodes present — exercises this; SiL testing with simulated perfect message delivery does not.

Failure mode injection is perhaps the most valuable unique capability of HiL. By controlling the simulated plant and the I/O interface, a HiL test engineer can inject failure modes that would be dangerous or impossible to create with real hardware: a sensor signal that drifts out of range at a controlled rate, an actuator that fails to respond, a power supply voltage that sags during a high-current transient, a communication bus that loses messages for 50ms. The unit under test must detect these faults, respond correctly, and transition to degraded modes — and HiL is the only environment where these scenarios can be tested safely and repeatably before the first physical system integration.

HiL in the certification evidence chain

For safety-critical systems subject to formal certification — DO-178C for avionics software, ISO 26262 for automotive, IEC 62061 for machinery, IEC 61508 for general functional safety — HiL testing occupies a specific and formally defined role in the verification evidence chain.

DO-178C requires that software requirements are verified by analysis or test, that test cases cover normal and abnormal range inputs, and that structural coverage of the software is measured. HiL testing contributes to requirements-based testing at the system level: test cases are derived from system requirements in the SysML architecture model, executed against the production software on the real hardware in a realistic plant environment, and results are documented as verification evidence. The traceability chain from SysML requirement to HiL test case to test result is the documentary backbone of the certification submission.

ISO 26262 for automotive functional safety distinguishes between software-level verification (unit tests, integration tests, SiL tests) and system-level validation (HiL tests, vehicle tests). The Automotive Safety Integrity Level (ASIL) of the function determines the required coverage of HiL testing — ASIL D functions, the highest safety level, require comprehensive HiL testing with documented fault injection results demonstrating correct behavior of the safety mechanisms.

In both domains the certification authority — FAA/EASA for aviation, TÜV/DEKRA for automotive — reviews not just the test results but the test coverage rationale: how do the test cases selected for HiL testing provide evidence that the system meets its requirements under all relevant operating conditions, including off-nominal conditions, boundary conditions, and failure modes? The selection of HiL test cases is a systems engineering activity, not just a test engineering activity, because it requires reasoning about the relationship between test coverage and requirement satisfaction — which is precisely what the MBSE architecture model is designed to support.

How HiL connects to the MBSE knowledge graph

The integration between HiL testing and the MBSE model is where the verification evidence management challenge lives. A mature program may run thousands of HiL test cases across multiple configurations of the plant simulation, multiple software versions, and multiple hardware revisions. Each test case needs to be traceable to the requirements it exercises, the test configuration it used, the software version under test, and the pass/fail result with any anomalies noted.

In the traditional document-based approach this traceability lives in test reports, requirements traceability matrices, and configuration management records that are maintained separately and reconciled manually. The consequence is the familiar pathology: a requirement is added to the SysML model late in the program, no one checks whether existing HiL test cases cover it, and the gap is discovered during a certification audit.

In the knowledge graph pattern you have architected across multiple domains, this traceability is structural. The HiL test case is a node connected by typed edges to the SysML v2 requirement it exercises (retrieved from the repository via the REST API), the plant model configuration used (FMU versions, parameter values), the software build under test, and the test result (pass/fail with timestamp and operator identity). A query against this graph can answer: "Which requirements have no HiL test case that has passed in the last six months against the current software build?" — the configuration management query that catches the gap before the certification audit does.

The Circuit Breaker pattern applies to HiL test configuration in the same way it applies to co-simulation setup. Before a HiL test run executes, the test configuration — which signals are injected, what plant model is loaded, what failure modes are scheduled — can be validated against the SysML v2 interface declarations. A test that injects a temperature signal in Fahrenheit to a controller that expects Celsius will produce a result that looks like a pass (the controller responds to the signal) but tests the wrong behavior. Grounding the test configuration against the architecture model's formally typed interface declarations catches this class of error before execution rather than during post-test analysis.

The Mean Time Between Hallucinations metric, translated to the HiL verification context, becomes the rate at which test configurations that were not validated against the current architecture model produce test results that later prove to be testing the wrong behavior — the HiL equivalent of an LLM confidently asserting something that the ground truth graph contradicts. The mechanism is identical: confident output from a system (the HiL test) that has drifted from its grounding in authoritative specifications (the SysML requirements and interface declarations). The remedy is identical: live, queryable, version-aware connections between the test configuration and the authoritative source of truth, with automated checks that trip before incorrect configurations execute.